

claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf\_a2b97cff-dcd\agent-af3e56fa37f9e2f74.jsonl`
- SHA-256: `d94b1c1f6cc442dcb8413e7ba515840b29569aaf807ba1ddccf3d432f3f9e26e`
- Source modified: `2026-06-21T13:56:24+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf\_a2b97cff-dcd`
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`

Transcript

1. user (2026-06-21T13:52:09.115Z)

Research how comparable LOCAL AI/CV/video apps ship "batteries included" and extract CONCRETE, copyable patterns: Ollama, LM Studio, ComfyUI, AUTOMATIC1111 / SD-Forge, Fooocus, Jan, GPT4All, faster-whisper/whisperX GUIs, and especially FRIGATE NVR (ffmpeg + ML detectors + web UI + Docker -- closest analog). For each note: distribution channel, how they handle the GPU/torch + ffmpeg + model-weights problems, install UX, and update mechanism. Conclude with the 2-3 patterns that best fit OUR app for a non-tech Windows user. Use web search heavily.

Return the RESEARCH schema. Context about our app:

PROJECT: a local, AI-powered badminton/racket-sports HIGHLIGHT INDEXER + rally detector.
Engine repo (PRIMARY): C:/Users/avidu/Projects/badminton-highlight-indexer (Python, FastAPI + SQLite + ffmpeg + GPU CV/AI; paid Gemini = cloud AI).
Sibling repos: C:/Users/avidu/Projects/khelsutra (the web/ React SPA + site/ marketing), C:/Users/avidu/Projects/rally-annotator (VLC plugin, MIT), C:/Users/avidu/Projects/sports-data-collector, C:/Users/avidu/Projects/sports-obsreport.

GOAL of the plan we are building: bake the engine into a DOWNLOADABLE, "batteries-included" Python APP that a user can download and run WITHOUT sourcing the repo (no git clone / build-from-source). It must spin up a local WEB SERVICE that runs (a) INFERENCE (video -> rallies -> highlight reel), (b) TRAINING (BYO model on their own labelled data), and (c) RALLY ANNOTATION if needed -- runnable BOTH locally and against CLOUD services. Preserve every already-completed end-to-end CUJ.

KEY FACTS the orchestrator already verified:

- pyproject.toml: hatchling build, namespace package (no backend/__init__.py), packages=["backend","training"], console script: indexer = backend.main:main. requires-python >=3.10.
- torch/torchvision are DELIBERATELY NOT in dependencies -- their CUDA/CPU wheels need --index-url (PEP621 cannot express). Optional extras: gpu(empty, docs-only), auth(PyJWT), reporting(git+ssh obsreport), storage-s3(boto3), storage-gcs(google-cloud-storage). google-genai is a hard dep.
- Server today: python -m backend.main --server --port 8000 . Worker: python -m backend.worker {infer|train} . CLI single-shot: python -m backend.main --video-path ... --segmenter {gemini|yolo_hybrid|wasb_hybrid|motion}.
- THE HARD PACKAGING PROBLEM: native WASB shuttle model runs in a SEPARATE Python env (conda, torch 1.11+cu113) while the main engine uses torch 2.6+cu124. ffmpeg/ffprobe are system deps. Model weights (wasb_badminton_best.pth.tar) live in a GCS bucket.
- Decoupled-compute SEAM already built (M1-M8 merged, default-OFF): deployment.profile + compute_target on AppConfig; backend/config/presets.py (PROFILE_PRESETS: truly-local / cloud-serving / cpu-mock); backend/storage/* (local/gcs/s3/gdrive StorageBackend registry); backend/compute/* (ComputeBackend registry, CloudRunCompute dispatches a Cloud Run Job); backend/config/startup.py per-profile checks; backend/billing.py cost ledger.
- The web SPA lives in a DIFFERENT repo (khelsutra/web, Vite/React/TS/Tailwind, i18n en+kn+hi). The engine has a likely-DEAD backend/api/static. Bundling the UI is a cross-repo concern.

- Licensing guardrail (NON-NEGOTIABLE): every dependency -- code, data, AND weights -- must be MIT/Apache-2.0/BSD only. Paid LLMs = inference/serving only, never training data. Any NEW packaging dependency must also be license-clean.
- A prior docs/DESKTOP_APP_PLAN.md exists but was SUPERSEDED by owner decision D10: deliver a "self-hosted webserver + UI the user spins up" (NOT a desktop app).
kshelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md is the live appliance design.
Always ground claims in file:line or a doc path. Read the actual files; do not assume.

2. assistant (2026-06-21T13:52:12.625Z)

I'll research how comparable local AI/CV/video apps ship "batteries included." Let me first ground myself in our app's actual packaging surface, then do heavy web research.

3. user (2026-06-21T13:52:13.878Z)

```
1      # PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).
2      #
3      # Build backend: hatchling. `backend/` is an intentional NAMESPACE package
4      # (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same
5      # reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's
6      # auto-detection would otherwise miss the un-__init__'d `backend` root.
7      #
8      # torch/torchvision are deliberately NOT listed in [project.dependencies]:
9      # their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url),
10     # which PEP 621 cannot express. Install them separately ? see [project.optional-
dependencies]
11     # `gpu` (documentation only) and the README. CI installs CPU torch on its own line.
12     [build-system]
13     requires = ["hatchling"]
14     build-backend = "hatchling.build"
15
16     [project]
17     name = "badminton-highlight-indexer"
18     version = "0.1.0"
19     description = "AI-assisted rally segmentation and highlight indexing for racket sports."
20     readme = "README.md"
21     requires-python = ">=3.10"
22     license = { text = "MIT" }
23     authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
24     keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
25
26     # Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).
27     dependencies = [
28         "fastapi>=0.111.0",
29         "uvicorn>=0.30.1",
30         "opencv-python>=4.9.0.80",
31         "numpy>=1.26.4",
32         "pydantic>=2.7.1",
33         "jinja2>=3.1.4",
34         "python-dotenv>=1.0.0",
35         "google-genai>=2.4.0",
36         "supervision>=0.21.0,<0.30.0",
37     ]
38
39     [project.optional-dependencies]
40     # Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).
41     dev = [
42         "pytest",
43         "pytest-cov",
44         "httpx",          # fastapi TestClient transport
45         "ruff",
46         "mypy",
47     ]
48
49     # Per-video observability reports (SOFT dependency: the pipeline degrades to a
```

```

50 # no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
51 # fetched over SSH; if you lack access, simply omit this group ? reporting
52 # tests skip via pytest.importorskip. For local dev against a sibling checkout:
53 # pip install -e ../sports-obsreport --no-build-isolation
54 reporting = [
55     "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
56 ]
57
58 # `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
59 # CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
60 # express, so they are NOT declarable here. Install them out-of-band instead:
61 # # CUDA 12.4 box:
62 # pip install torch>=2.12.0 torchvision>=0.27.0 --index-url
https://download.pytorch.org/whl/cu124
63 # # CPU-only box (also what CI uses):
64 # pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
65 gpu = []
66
67 # Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs NONE of
68 # these. Cloud SDKs are imported lazily, so `import backend.storage` works without them;
69 # install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare
R2 +
70 # DO Spaces + MinIO via endpoint_url.) `gdrive://` needs NO extra ? it reads a locally
71 # MOUNTED Google Drive (Google Drive for Desktop) as plain files; see
backend/storage/gdrive.py.
72 storage-s3 = ["boto3>=1.34"]
73 storage-gcs = ["google-cloud-storage>=2.16"]
74
75 [project.scripts]
76 # `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
77 indexer = "backend.main:main"
78
79 [tool.hatch.metadata]
80 # The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
81 # reference*); hatchling requires opting in to allow those in metadata.
82 allow-direct-references = true
83
84 [tool.hatch.build.targets.wheel]
85 # `backend` is a namespace package (no __init__.py); list both import roots
86 # explicitly so an editable/wheel install exposes `import backend` and
87 # `import training` regardless of cwd.
88 packages = ["backend", "training"]
89

```

4. assistant (2026-06-21T13:52:20.026Z)

Let me read the key appliance/packaging docs in our repos to ground the "fit for our app" conclusion, while kicking off web research in parallel.

5. user (2026-06-21T13:52:21.710Z)

C:\Users\avidu\Projects\kheIsutra\docs\LOCAL_APPLIANCE_ARCHITECTURE.md

6. user (2026-06-21T13:52:22.017Z)

docs\DESKTOP_APP_PLAN.md

7. user (2026-06-21T13:52:30.459Z)

Web search results for query: "ComfyUI desktop installer Windows portable how it bundles python torch update mechanism"

Links: [{"title": "Manual Installation ? Local Self-Hosted - ComfyUI", "url": "https://docs.comfy.org/installation/manual_install"}, {"title": "ComfyUI

```
Installation Tutorial ? Creepybits", "url": "https://zanno.se/comfyui-installation-tutorial/"}], [{"title": "Installation and Setup | Comfy-Org/ComfyUI | DeepWiki", "url": "https://deepwiki.com/Comfy-Org/ComfyUI/1.1-installation-and-setup"}, {"title": "GitHub - Tavris1/ComfyUI-Easy-Install: Portable ComfyUI installer for Windows, macOS and Linux with EZI Desktop app ? Nvidia GPU support ? Pixaroma Community Edition", "url": "https://github.com/Tavris1/ComfyUI-Easy-Install"}, {"title": "Installation and Setup | comfyanonymous/ComfyUI | DeepWiki", "url": "https://deepwiki.com/comfyanonymous/ComfyUI/11.1-installation-and-setup"}, {"title": "ComfyUI portable is using its own PyTorch version that cannot be updated · Issue #6197 · Comfy-Org/ComfyUI", "url": "https://github.com/comfyanonymous/ComfyUI/issues/6197"}, {"title": "GitHub - Comfy-Org/ComfyUI: The most powerful and modular diffusion model GUI, api and backend with a graph/nodes interface. · GitHub", "url": "https://github.com/Comfy-Org/ComfyUI"}]]
```

Based on the search results, here's what I found about ComfyUI's desktop installer, Windows portable version, and how it bundles Python, PyTorch, and handles updates:

Windows Portable Bundle

The Windows portable package is a pre-configured standalone build that requires no installation and bundles a minimal embedded Python environment and pre-installed dependencies, making it fully portable. It's a compressed folder containing everything needed, and the file is a bit over 1GB.

Python and PyTorch Bundling

The current ComfyUI installation comes with Torch 2.9 for CUDA 13.0 pre-installed. The portable version uses an embedded Python installation (referenced as `python\_embeded` in the documentation) rather than relying on system-level Python, which helps avoid conflicts with other dependencies on the system and prevents polluting the system-level Python environment.

Update Mechanism

The portable version supports self-updates via the update.py script under .ci/update_windows/, which uses pygit2 for Git operations. Additionally, the update mechanism includes pre-configured batch files in the update/ directory that invoke python.exe to upgrade torch and other requirements from requirements.txt.

Desktop Application

The desktop application is the easiest installation method, providing a GUI installer with automatic dependency management and features automatic updates with an integrated model and workflow manager.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

8. user (2026-06-21T13:52:38.456Z)

Web search results for query: "Frigate NVR Windows install Docker how it handles ffmpeg ML detectors models"

Links: [{"title": "Frigate + Reolink + NVIDIA Setup Guide (2026): Docker, go2rtc & TensorRT", "url": "https://corelab.tech/setupfrigate/"}, {"title": "GitHub - blakeblackshear/frigate: NVR with realtime local object detection for IP cameras · GitHub", "url": "https://github.com/blakeblackshear/frigate"}, {"title": "Frigate | Seeed Studio Wiki", "url": "https://wiki.seeedstudio.com/ODYSSEY-X86J4105-Frigate/"}, {"title": "Frigate Docker Compose / Portainer - I.T. Bacon", "url": "https://itbacon.com/2024/05/08/frigate-docker-compose-portainer/"}, {"title": "Releases · blakeblackshear/frigate", "url": "https://github.com/blakeblackshear/frigate/releases"}, {"title": "Gun detection with Frigate NVR on R2000 | Seeed Studio Wiki", "url": "https://wiki.seeedstudio.com/gun_detection_with_frigate_nvr_on_r2000/"}, {"title": "Installation | Frigate", "url": "https://docs.frigate.video/frigate/installation/"}, {"title": "Getting"}]

```
started | Frigate", "url": "https://docs.frigate.video/guides/getting_started/"]}]
```

Let me search for more specific information about Windows Docker support for Frigate.

```
Links: [{"title": "Frigate on Docker, in Windows Subsystem for Linux (WSL) ·
blakeblackshear/frigate · Discussion #4375", "url": "https://github.com/blakeblackshear/frigate/di
scussions/4375"}, {"title": "Installation | Frigate", "url": "https://docs.frigate.video/frigate/ins
tallation/"}, {"title": "frigate/docs/docs/frigate/installation.md at dev · blakeblackshear/frigat
e", "url": "https://github.com/blakeblackshear/frigate/blob/dev/docs/docs/frigate/installation.md?
plain=1"}, {"title": "blakeblackshear/frigate - Docker
Image", "url": "https://hub.docker.com/r/blakeblackshear/frigate"}, {"title": "How to Self-Host
Frigate with Docker Compose |
selfhosting.sh", "url": "https://selfhosting.sh/apps/frigate/"}, {"title": "Frigate host on windows
11 mini pc - external GPU RTX 3060 - Share your Projects! - Home Assistant
Community", "url": "https://community.home-assistant.io/t/frigate-host-on-windows-11-mini-pc-
external-gpu-rtx-3060/876921"}, {"title": "Frigate From Scratch guide -
karl", "url": "https://karlquinsland.com/frigate-install/"}, {"title": "GitHub -
blakeblackshear/frigate: NVR with realtime local object detection for IP cameras ·
GitHub", "url": "https://github.com/blakeblackshear/frigate"}]}
```

Let me search for more specific information about how Frigate handles ML models and detectors.

```
Links: [{"title": "Object Detectors |
Frigate", "url": "https://docs.frigate.video/configuration/object_detectors/"}, {"title": "[Detector
Support]: onnx on tensorrt image v0.16 beta4 · blakeblackshear/frigate · Discussion #19140", "url
": "https://github.com/blakeblackshear/frigate/discussions/19140"}, {"title": "frigate/docs/docs/co
nfiguration/object_detectors.md at dev · blakeblackshear/frigate", "url": "https://github.com/blak
eblackshear/frigate/blob/dev/docs/docs/configuration/object_detectors.md"}, {"title": "Guide how
to create and use YOLO-NAS ONNX models with Frigate 0.17 TensorRT Image (GeForce GPU) ·
blakeblackshear/frigate · Discussion
#21858", "url": "https://github.com/blakeblackshear/frigate/discussions/21858"}, {"title": "Frigate
- Leo's Notes", "url": "https://leo.leung.xyz/wiki/Frigate"}, {"title": "Frigate + Reolink + NVIDIA
Setup Guide (2026): Docker, go2rtc &
TensorRT", "url": "https://corelab.tech/setupfrigate/"}, {"title": "[Config Support]: Setting up
frigate with nvidia tensor problem · blakeblackshear/frigate · Discussion
#22278", "url": "https://github.com/blakeblackshear/frigate/discussions/22278"}, {"title": "Frigate
0.16.1 TensorRT ONNX Model Proper Ingestion", "url": "https://ioritro.com/blog/2025-09-01-frigate-
onnx-detection-model/"}]}
```

Based on my search results, here's a comprehensive overview of Frigate NVR's installation on Windows, Docker, and how it handles ffmpeg and ML detectors:

Windows Installation with Docker

Windows is not officially supported, but some users have had success getting it to run under WSL or Virtualbox. Getting the GPU and/or Coral devices properly passed to Frigate may be difficult or impossible. However, some users have been able to get Frigate running with Nvidia GPU Tensor, as well as the Google Coral USB on Windows via WSL 2 and Docker Desktop.

Frigate runs best with Docker installed on bare metal Debian-based distributions. For ideal performance, Frigate needs low overhead access to underlying hardware for the Coral and GPU devices.

FFmpeg Integration

The low-resolution feed is routed to the FFmpeg Decoder, where it is decoded using Hardware Acceleration (HW Accel) to minimize CPU usage. ffmpeg has been updated to version 7. For hardware acceleration, if you are using hardware acceleration for ffmpeg, you may need to use the Full Access variant of the App. This is because the Frigate App runs in a container with limited access to the host system. The Full Access variant allows you to disable Protection mode and give Frigate full access to the host system.

ML Detectors and Models

Frigate supports multiple detector types and models:

****Detector Types:**** Frigate recommends using a USB Google Coral for this task, but you may also choose to use a NVIDIA GPU with TensorRT (supported on post-Pascal GPUs only). Additionally, OpenVINO will automatically be detected and used as a detector in the default Frigate image when a supported ONNX model is configured. Nvidia GPUs will automatically be detected and used as a detector in the -tensorrt Frigate image when a supported ONNX model is configured. TensorRT can run on Jetson devices, using one of many default models. ONNX: TensorRT will automatically be detected and used as a detector in the -tensorrt-jp6 Frigate image when a supported ONNX model is configured.

****Model Conversion:**** Once built, copy the .onnx file to your /config/model_cache folder. Frigate will automatically convert this to a TensorRT engine on the first boot. The replacement for TensorRT, the ONNX detector, no longer ships with prebuilt models?even if you're using the 0.16.x-tensorrt images. In other words: you now have to provide your own models.

****Recommended Models:**** YOLO-NAS or YOLOv9 are recommended as they provide superior performance and accuracy compared to the models supported by TensorRT.

****Multiple Detectors:**** When using many cameras one detector may not be enough to keep up. Multiple detectors can be defined assuming GPU resources are available.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

9. assistant (2026-06-21T13:52:42.330Z)

Let me read the appliance docs and continue web research in parallel.

10. user (2026-06-21T13:52:43.751Z)

```
1      # Single-box appliance ? UI-powered rally indexer + reel generator (KhelSutra Studio)
2
3      > **Status:** `DRAFT` ? owner-gated (auth choice, retention, cost ceiling, GPU host, P5
bridge) . **Owner:** Avi . **Created:** 2026-06-20 . **Last updated:** 2026-06-20
4      > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5      > **Tracking anchors:** $9 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in `NEXT_STEPS.md` + memory `current-state` once work starts.
6      > **Relation:** peer-of `PER_RALLY_SELECTION.md` / `REQUEST_ACCESS_FORM.md` in this
repo; **consumes** the engine's decoupled `StorageBackend` + `python -m backend.worker` seams
and the FastAPI loop; **extends** the engine's `HOSTING_PLAN.md` gating + `UI_PROPOSAL.md` alpha
screens. Engine-side changes here are **proposals to coordinate** with the active
decoupled/worker sessions ? not written unilaterally.
7      > **Honesty note:** every engine-fact claim is tagged `[verified]` (read against engine
source this session), `[design]` (proposed here), or `[researched]` (needs sign-off). Not
legal/financial advice.
8
9      ---
10
11     ## 0. TL;DR
12     **KhelSutra Studio** turns the engine from a CLI into a **single-box, UI-powered rally
indexer + reel generator**: the khelsutra `web/` React SPA, served behind an **edge auth gate**,
drives the **existing** engine FastAPI loop (ingest ? index ? pick rallies ? reel ? download) on
**one machine**, with `StorageBackend=local` for the MVP. It is the **single-box deployment
profile of the just-shipped decoupled architecture** ? storage=local + compute=local is the
degenerate collapse of the same storage/worker split.
13
14     The **single most important caveat:** the engine has **zero app-layer auth and CORS
`***` `[verified backend/main.py:42]`, so safety is structural **only** because the engine stays
bound to `127.0.0.1:8000` `[verified backend/main.py:638]`, a firewall blocks inbound `8000`,
and a reverse proxy/tunnel is the sole authenticated ingress ? **the gate, not the app, is the
tenancy boundary.**
```

15

```

16     Two honest hardware truths: (1) the current droplet is CPU-only ? real detection
there is Gemini (cloud) or stub (CPU mock); real local-GPU detection needs a GPU
host (native WASB, in-flight). (2) The owner priority is to stand up a GPU host ASAP ?
provider-agnostic (DO GPU droplet / GCE GPU VM / Cloud Run+GPU) ? but that runs in parallel
with the UI MVP, which ships on Gemini today.
17
18     ## 1. Problem & goal
19     The decoupled engine just shipped a `StorageBackend` ABC + a `python -m backend.worker
{infer|train}` dispatcher `[verified backend/storage/, backend/worker/___main__.py]`, and the
FastAPI loop already covers `process ? jobs ? query ? compile ? highlights` `[verified
backend/main.py]`. What's missing is a console that lets a non-CLI operator run the whole
loop on a single box, honestly, behind a gate.
20
21     Headline CUJ (owner's words): a hosted website where I pick an input ? a
Google Drive file, a bucket file (`s3:///gs://`), or a local upload ? it lands in the
bucket (DO Spaces / GCS) or locally ? I generate highlights on a GPU-enabled host
running the real e2e ? I pick rallies, watch, and download ? completely from the UI.
22
23     Outcome ("good"): an operator signs in, picks a source, watches it index, curates
rallies, downloads a reel ? on the live CPU droplet today (Gemini/stub), and on a GPU host
later (native WASB) with no UI fork.
24
25     Read to get current: `PER_RALLY_SELECTION.md` ($3/$4/$10 contract anchors), engine
`HOSTING_PLAN.md` (gating), `DECOUPLED_COMPUTE/HANDOFF.md` (what's built),
`VIDEO_LOCALITY_MODEL.md` (privacy/retention).
26
27     ## 2. Decisions locked
28     | # | Decision | Source / date | Implication |
29     |---|---|---|---|
30     | D1 | Reverse-proxied single origin; engine stays `127.0.0.1:8000` | `[verified
main.py:638]`; 2026-06-20 | CORS `` is moot from the browser ? the proxy is the boundary |
31     | D2 | Edge auth, zero engine auth code | `HOSTING_PLAN.md`; 2026-06-20 | Caddy
basicauth OR cloudflared + Cloudflare Access |
32     | D3 | MVP drives the in-process FastAPI worker loop, `storage=local` | `[verified
job_worker.py:37, local.py:33]`; 2026-06-20 | bucket worker = reserved scale path, out of MVP |
33     | D4 | Embed RallyPicker from `PER_RALLY_SELECTION.md` verbatim | PR [#11];
2026-06-20 | the console is a superset; do not redesign the picker |
34     | D5 | Console lives in the khelsutra web/ React scaffold | `web/README.md`;
2026-06-20 | Cloudflare-Pages-style root, not engine static |
35     | D6 | Compute placement is a setting, rendered from `GET /api/capabilities` |
`[design]`; 2026-06-20 | UI offers ONLY what the box can honestly do |
36     | D7 | Ingest is a source menu (local upload · bucket URI · Drive) normalizing to
one video object | `[verified backend/storage/ref.py parse_uri]`; 2026-06-20 | Drive is a
stub today ? scoped honestly ($4.2) |
37     | D8 | GPU host is provider-agnostic (DO droplet / GCE VM / Cloud Run+GPU); storage
stays the bucket | Owner 2026-06-20 | compute and storage are separate planes; no provider lock-
in |
38     | D9 | Observability is first-class ? structured logs on every new path + a DoD item
| Owner 2026-06-20 | see $5; verified during CUJ replay |
39     | D10 | Delivery = a self-hosted webserver + UI the user spins up (NOT a desktop
app); the user points at local storage + GPU/CPU OR cloud locations and the UI drives the
tool | Owner 2026-06-20 | supersedes `DESKTOP_APP_PLAN.md`; one artifact (this appliance)
runs on the owner box, a droplet, or a GPU host ? same UI, compute/storage are settings |
40
41     ## 3. Foundation ? what the engine gives us today `[verified this session unless
noted]`
42     - FastAPI loop: `POST /api/process` (202 + `job_id`) ? poll `GET /api/jobs/{id}` ?
`POST /api/query` ? `POST /api/compile` ? `GET /api/highlights/{video_id}/{filename}`
(FileResponse) `[main.py]`. Chunked upload exists ? `POST /api/upload/init` · `PUT
/api/upload/{id}/part/{i}` · `POST /api/upload/{id}/complete` · `DELETE /api/upload/{id}`
`[verified uploads.py:64,89,123,147]` ? but the bundled `app.js` never calls them (it only
hits `/api/process` + `/api/compile` `[verified app.js:88,373]`). So browser upload is client-
wiring only, no engine change.
43     - In-process worker: one module-global `ThreadPoolExecutor(max_workers=1)`
`[verified job_worker.py:37]` ? strictly one job at a time; the DB `jobs` table is the

```

```

clock; `fail_stale_jobs` sweeps queued/running on boot. Gemini chunk concurrency is a
**separate** knob `indexing.ai_max_workers` (**default 5** `[verified models.py:183]`; set to 1
to serialize) ? it is **not** already 1.
44 - **Storage:** `storage.backend` default = "local" `[verified models.py:298]`;
`LocalStorage.fetch_to_local` is **identity** `[verified local.py:33]`, `put` no-ops on
`src==dst` ? so on a single box the worker's pull?pipeline?push **collapses to read-local/write-
local**. URI scheme `parse_uri` `[verified ref.py]: bare/`local://` ? local; `s3://`
(AWS/R2/**DO Spaces**/MinIO, by endpoint) ; `gcs://`/`gs://` ; `gdrive://` ? **follow-up stub
(emulated only)**. Credentials never in config ? boto3 default chain / `~/aws` `[verified
s3.py]`.
45 - **Compute seam:** `indexing.detector_impl` ? {`auto`=WSL, `native`=Linux-native,
`stub`=CPU mock}; default "auto" `[verified models.py:129]`, resolved in
`trajectory_hybrid._resolve_runner` `[verified :76-85]`. **Native WASB is in-flight on branch
`feat/native-wasb-runner`:** `native_wasb_runner.py` **exists** `[verified]`;
**`fusion_hybrid._build_native_runner` returns `NativeWasbRunner`** `[verified
fusion_hybrid.py:86]` while **`trajectory_hybrid._build_native_runner` still raises** `[verified
:57-63]`. Acceptance gate = **owner byte-parity** (WSL CSV == native CSV) on a real GPU box ?
not exercisable on the droplet or in the offline suite. Treat native as **not production-
verified**.
46 - **Worker CLI:** `python -m backend.worker {infer|train}` pulls a `--video-uri` ?
pipeline ? pushes `outputs/<video_id>/{intervals.csv,report.json}` `[verified
worker/__main__.py:88,149]`. The validated GPU-free/key-free combo: `detector_impl=stub` +
`--set indexing.skip_ai_handoff=true` + `storage.s3`.
47 - **Gating:** cloudflared Tunnel (outbound-only, no inbound ports) + Cloudflare Access
email-OTP; backend trusts `Cf-Access-Authenticated-User-Email` `[HOSTING_PLAN.md]`. `GET
/api/health` **now EXISTS** (engine M1, [#268]) ? {status, sport, default_segmenter}`
[verified main.py:127, corrected 2026-06-21 ? was "does not exist"]`.
48 - **Droplet reality:** `168.144.126.176`, **CPU-only, NO GPU**, region `blr1`; co-hosts
the marketing site (`/srv/khelsutra-site`, node `:8787`) **and** the engine (`~/rally`); ran the
full train+infer e2e against DO Spaces with the **GPU mocked** `[SETUP_NEW_MACHINE.md,
DECOUPLED_COMPUTE/HANDOFF.md]`. Shared testbed: MinIO `9000/9001` + GCS emulator `9023` also run
there.
49 - **Absent (net-new):** `GET /api/capabilities`, `GET /api/videos` (only single-row
`get_video(id)`), `GET /api/reels/{video_id}`, any source/proxy streaming endpoint, any
**Dockerfile**, any per-video **cost ledger** `[verified ? grep returns none]`.
50
51 ## 4. Design / architecture
52
53 ### 4.1 Topology & request path
54 ONE box: **edge gate** (cloudflared+Access or Caddy basicauth) ? **reverse proxy /
static host** serving the `web/` SPA bundle and proxying `/api/` ? engine **FastAPI
127.0.0.1:8000** ? **LocalStorage** ? **detector seam** (Gemini/stub today; native WASB on a
GPU host). The firewall blocks inbound `8000` + the test ports `9000/9001/9023`. The marketing
site stays a **separate** systemd unit on a **separate** subdomain.
55
56 ```
57 PUBLIC INTERNET
58 ? (1) EDGE GATE ? the ONLY ingress; auth lives HERE, not in the app
59 ? cloudflared Tunnel + Cloudflare Access (no inbound ports) OR Caddy :443 +
basicauth
60 ?
61 ?????????????????????????????????????????????????????????????????????????????????
62 ? ONE BOX firewall BLOCKS inbound 8000 + 9000/9001/9023 ?
63 ? (2) reverse proxy / static host (Caddy/cloudflared) ?
64 ? /, /assets/* ? web/ React SPA /api/*, /api/highlights/* ? :8000 ?
65 ? ? ONE ORIGIN to the browser ? engine CORS '*' is irrelevant ?
66 ? ? ?
67 ? (3) ENGINE FastAPI (host=127.0.0.1:8000, HARD-bound [main.py:638]) ?
68 ? control: jobs table + ThreadPoolExecutor(max_workers=1) drain ?
69 ? data: uploads.py chunked upload . /api/compile . /api/highlights ?
70 ? ? ?
71 ? (4) STORAGE = LocalStorage (default) fetch=identity . put=no-op(src==dst) ?
72 ? ? ?
73 ? (5) COMPUTE seam: gemini (cloud, key) . stub (CPU mock) [droplet] ?
74 ? native WASB (GPU host) . auto (owner Win+WSL) ?

```



```

75      ?????????????????????????????????????????????????????????????????
76      RESERVED SCALE PATH (out of MVP): console "compute target" selector ? storage=s3 (DO
Spaces)
77      ? SEPARATE GPU host runs `python -m backend.worker infer --video-uri s3://?` ?
outputs/<vid>/ back
78      (needs a NET-NEW orchestration endpoint reflecting outputs into the jobs/poll
contract)
79      ...
80
81      ### 4.2 Ingest sources (the headline) ? one menu, one normalized video object
82      All sources normalize to ***"a video the pipeline can read"*** ? a local path (MVP) or a
`StorageRef` (scale):
83
84      | Source | Path today | Status |
85      |---|---|---|
86      | **Local upload** | SPA chunks the file ? `max_part_mb` via the existing
`/api/upload/init\|part\|complete` `[verified uploads.py]` ? server path ? `POST /api/process`.
(Scale: also `StorageBackend.put` to `s3://`/`gcs://`. ) | **wire client only** (endpoints exist)
|
87      | **Bucket URI** (`s3://`, `gs://`) | **`POST /api/process` accepts the bucket URI
directly** (resolve ? fetch to scratch ? same pipeline ? API DB) `[verified main.py:159,274,283
? corrected 2026-06-21; was "net-new"]`; the worker CLI also fetches `--video-uri`. So the
**engine endpoint is NOT net-new** ? only the SPA call that POSTs the URI + polls `/api/jobs`
is. *(Default single-user mode; HOSTED mode w/ `allowed_video_root` correctly rejects a bucket
URI 400.)* | **engine: DONE (verified); SPA ingest call: design (= B-P2)** |
88      | **Google Drive** (`gdrive://`) | `parse_uri` recognizes it, but the gdrive backend is
an **emulated stub** `[verified ref.py, DECOUPLED_COMPUTE research]` | **stub ? roadmap;** MVP
requires a Drive?bucket pre-step; do **not** present Drive as live until the backend is real |
89
90      ### 4.3 Compute placement matrix (one UI, surfaced from `/api/capabilities`)
91      | Placement | Runs | Needs | Real/mock | Honest caveat |
92      |---|---|---|---|---|
93      | **CPU droplet ? gemini** (MVP target) | `segmenter=gemini`, in-proc worker |
`GEMINI_API_KEY`; network; **no GPU** | **REAL** | Cloud brain: uploads ?1280px chunks to Google
? **must be in consent copy** ; **no per-video cost ceiling** today |
94      | **CPU droplet ? stub** | trajectory hybrid + `detector_impl=stub`+`skip_ai_handoff` |
nothing | **MOCK** | not real tracking ? UI must label "stub (CPU mock ? not real detection)";
demo/regression only |
95      | **CPU droplet ? motion** | `segmenter=motion` | nothing | REAL, low-quality | only no-
key/no-GPU real segmenter; never surface `server/receiver/events` (fabricated) |
96      | **GPU host ? native WASB** (in-flight) | `wasb`+`detector_impl=native` ?
`NativeWasbRunner` (in-proc torch) | a **GPU host ? DO GPU droplet / GCE GPU VM / Cloud
Run+GPU** ; `WASB_WEIGHTS` | **REAL** , on-device | partially wired (fusion builds it, trajectory
refuses); **owner byte-parity gate** ; **Cloud Run/GCE need a containerized worker ? no
Dockerfile exists, and WASB's torch-1.11 conda env makes the image non-trivial** |
97      | **Owner Win+WSL ? auto** | `wasb`+`detector_impl=auto` (WSL2+conda) | Windows+WSL2+GPU
| REAL (today) | Windows-only transport; the working real-GPU path for owner verification |
98      | **Remote GPU worker via bucket** (reserved) | console CPU + `storage=s3`; separate GPU
host runs `backend.worker infer` | bucket creds; a GPU host; a **net-new orchestration
endpoint** | REAL (remote) | FastAPI loop and worker CLI are **separate** today; "remote GPU as
a setting" is **design** ? selector shown **disabled** in MVP |
99
100     **Provider note:** **Cloud Run + GPU** (scale-to-zero, pay-per-job) is the strongest fit
for the engine's own **per-video-billing** economics (`DECOUPLED_COMPUTE` #246) ? gated on the
**containerization feasibility check** above.
101
102     ### 4.4 Auth / exposure (non-negotiable invariant)
103     No engine auth, by design; the gate is external. **Invariant (all three must hold):**
engine bound to `127.0.0.1` + firewall closes `8000` + edge gate up. Go-live checklist
`[HOSTING_PLAN.md:102-108]`: Access/basicauth ON; **rotate the chat-shared `GEMINI_API_KEY`** ;
set `security.allowed_video_root`; rate-limit; per-user quota.
104
105     ### 4.5 UI operator console (web/ SPA ? superset of RallyPicker)
106     - **S1 Ingest & Process** ? the source menu ($4.2) + a segmenter picker from
`/api/capabilities` ? `POST /api/process`.

```

107 - ****S2 Progress**** ? 3s poll of the free-text stage
(`hashing/validating/segmenting/finalizing`), a *****one job at a time***** notice, and the restart-failure terminal state.

108 - ****S3 Rally pick ? reel**** ? ****embed RallyPicker verbatim**** (`PER\_RALLY\_SELECTION.md`):
honor `min\_shot\_count`; never render motion `server/receiver/events` or a fake confidence meter.
****Per-rally is ONE screen of the console.****

109 - ****S4 Reels / History**** ? needs `GET /api/videos` + `GET /api/reels/{video\_id}` for
****reload-survival****.

110 - ****S5 Settings**** ? render `config` + `capabilities`; the compute/storage ****target selector shown DISABLED**** ("single-box: local") until the bucket-worker bridge lands.

111

112 **### 4.6 Reuse map**

113 ****Reused (no re-plumb):**** the full `/api` loop; the in-process worker; the chunked upload endpoints (un-called); LocalStorage default; the detector seam incl. the landing `NativeWasbRunner`; the ****RallyPicker**** design + anchors; the `web/` scaffold; the `HOSTING\_PLAN` gate; the `SETUP\_NEW\_MACHINE` runbook. ****Net-new:**** Caddyfile + systemd units; `GET /api/capabilities`; `GET /api/videos`; `GET /api/reels/{video\_id}`; `GET /api/health`; the SPA console screens; ingest-from-bucket wiring; a retention sweep; (reserved) the bucket-worker bridge.

114

115 **## 5. Observability ? logging & monitoring** `[required, first-class]`

116 Launch-readiness directive (owner 2026-06-20): ****logs and monitoring taken seriously, with a full log-analysis pass across every new code path.**** The engine already uses a centralized Python logger `[verified main.py:12-17]` ? carry that discipline into every new path.

117

118 - ****Correlation ID end-to-end.**** Thread the existing `job\_id` (from `/api/process`) ****plus**** an `upload\_id`/`request\_id` through ingest ? process ? jobs ? compile ? highlights, and into the worker for the bucket path, so ****one CUJ is traceable**** across UI ? FastAPI ? worker ? GPU host. Log it on every line.

119 - ****Structured logs.**** New endpoints/screens emit structured (JSON-able) logs at ****entry / result / error**** ? not just "it ran". The SPA logs client-side errors + failed fetches with the same correlation id.

120 - ****Loud failures, never silent drops.**** Carry the engine's ethos (#240 "surface failed chunks loudly") + RallyPicker's `min\_shot\_count`-warning rule: a partial/empty reel must ****never**** look complete; every dropped clip/chunk is logged + surfaced in the UI.

121 - ****Health + metrics.**** Ship the missing `GET /api/health` `[HOSTING\_PLAN.md:42]` and a thin metrics surface (jobs queued/running/failed, last error, disk free, per-video cost once a ledger exists) for the engine-mode banner + alerting.

122 - ****Aggregation for transient hosts.**** A scale-to-zero Cloud Run / GPU host is ****ephemeral**** ? logs must ship to a durable sink (the bucket or a log service), not die with the instance.

123 - ****Log-analysis pass before merge (DoD item).**** Every PR that adds a code path includes a check that the path logs meaningfully, and the ****logs are inspected during CUJ live-replay**** (§8 Layers 1?3). Ties [[feedback-launch-quality-bar]].

124

125 **## 6. Risk table**

Risk	Today	Mitigation
No app auth + CORS `` `[main.py:42]` open if any of {localhost-bind, firewall, gate} slips enforce all three; **LIVE posture test** that raw `:8000` is refused from outside	Test ports on shared droplet MinIO `9000/9001` + GCS emu `9023` firewall closes them	Retention `output/`+`uploads/` originals **never deleted** sweep / keep-proxy-only before any non-owner exposure
Surprise Gemini/GPU bill **no cost ceiling/ledger** per-video cap; `ai_max_workers` (default 5) tunable; `max_workers=1` job serialization	Native WASB unverified partially wired; owner-parity-gated treat as not-production-verified until owner byte-parity on a real GPU box	New engine endpoints `capabilities`/`videos`/`reels`/`health` don't exist coordinate engine PRs with active sessions; **branch from `origin/master`, re-verify HEAD, explicit `git add`**
Drive ingest overstated `gdrive://` is a stub don't present Drive as live; require Drive?bucket pre-step until real		

```

135 | No Dockerfile | runbook-only reproducibility | feasibility-check a worker image before
promising Cloud Run/GCE |
136 | Shared-checkout collisions | active worker sessions share the engine tree | the
branch-safety protocol on every engine-side PR |
137 | **Provisioning-credential leak** | a DO API token / cloud creds could leak if pasted
in chat or committed (cf. the **already-rotated Spaces key** exposed in chat,
`DECOUPLED_COMPUTE/HANDOFF.md`) | authenticate the CLI **on-box** (`doctl auth init` interactive
/ `gcloud` already authed) so the secret never enters chat or the repo; least-privilege token,
revoke after; creds 0600, never committed; **creating paid GPU infra is a confirm-first action
every time** |
138
139 ## 7. Honest limits ? what this does NOT do
140 Does **not** add multi-tenant identity/RLS (single-tenant alpha; uploads/outputs
global). Does **not** run real WASB on the CPU droplet (Gemini or stub only). Does **not** make
the remote-GPU bucket path work (design; selector disabled). Does **not** provide in-UI source
playback, job cancel, or a true progress bar (coarse free-text stage, poll-only). Does **not**
containerize (runbook; no Dockerfile `[verified]`). Does **not** verify native WASB (owner byte-
parity on a real GPU box, not this offline suite). A green SPA + contract tests prove the
**console plumbing**, not engine rally-detection quality (a separate track).
141
142 ## 8. CUJs & live-test plan `[required]`
143 **CUJs:** **CUJ-1** ingest (Drive/bucket/local) ? index ? reel ? download (core);
**CUJ-2** re-open a past match after reload; **CUJ-3** pick-by-query (deterministic filters +
honesty-hiding); **CUJ-4** settings/compute placement (truthful capabilities).
144
145 **Live-test plan ? 5 layers, every CUJ replayed LIVE throughout development:**
146 - **L1 ? Mock engine (hermetic, fast, default + CI).** A tiny FastAPI/MSW implementing
the **exact** contract (upload; `process` 202; `jobs` state machine incl. restart-failure;
`query` with a motion fixture for honesty-hiding; `config`; `capabilities` per box profile ?
CPU-no-key / CPU-with-key / GPU; `compile`; `highlights` tiny mp4; the new `videos`/`reels`).
The SPA drives **all 4 CUJs in <1s**, **replayed via the Claude Preview/browser tools** (start
dev server ? fill the ingest picker ? click through process/poll/RallyPicker ? assert the reel
link via snapshot/network). Catches reload-survival (CUJ-2), the `play_segments` field-name bug
(CUJ-3), capability honesty (CUJ-4).
147 - **L2 ? Engine-contract checks (real FastAPI, GPU-free).** Boot `python -m backend.main
--server` with `storage=local` + `detector_impl=stub` + `skip_ai_handoff` and assert the **mock
and the real engine agree shape-for-shape** (202 keys, jobs enum, `result.play_segments` schema,
compile `{status,filepath}`, highlights content-type). Every new endpoint is born with a mock
handler **and** a real-engine contract test **in the same PR**. Runs in CI.
148 - **L3 ? Real local e2e (the MVP substrate).** Through the gate on the droplet: CUJ-1
with `stub` (key-free) and `gemini` (real, short clip to bound cost); the **exposure-safety
posture test** (raw `:8000`/`:900x` refused from outside) as an assertion, not an assumption;
reuse the `parity.yml` dual-host pattern.
149 - **L4 ? CI regression (green gates merge).** `npm test` + a stub-engine contract job +
headless browser-replay CUJ scripts; one small PR per tracker row off `origin/master`, **no
stacking**; coverage at/above the floor; **logs inspected during replay** ($5).
150 - **L5 ? Owner hardware e2e (handed off).** Native WASB **byte-parity** on a real GPU
box; real-4K **golden-corpus** runs (Gemini + native); Modern-Standby/`cv2`-vs-`ffmpeg`
behavior; a multi-GB browser upload. Reported back into $9, stated plainly as **not verified**
in the offline suite.
151
152 ## 9. Deliverables & progress tracker ? **source of truth**
153 Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**,
branched from `origin/master`, no stacking; CI green gates merge; coverage ? floor; each PR
ships its tests + meaningful logs ($5).
154
155 | ID | Deliverable | Depends on | Gated? | Status | PR |
156 |----|-----|-----|-----|-----|----|
157 | T0 | This design doc + `docs/README.md` + `NEXT_STEPS.md` pointers | ? | owner | ? |
(this PR) |
158 | T1 | Mock engine + capability fixtures + browser-replay CUJ scripts (L1) | T0 | no | ?
| ? |
159 | T2 | Appliance shell: Caddyfile + systemd units; firewall; **edge auth**; rotate key
(P1) | ? | owner | ? | ? |
160 | T3 | L3 live posture test (raw `:8000`/`:900x` refused from outside) | T2 | no | ? | ?

```

```

|
161 | T4 | S1 ingest (local upload wired) + segmenter picker | T1 | no | ? | ? |
162 | T5 | S2 progress (poll + one-job + restart-failure) | T4 | no | ? | ? |
163 | T6 | S3 embed **RallyPicker** + compile + download | T4 | no | ? | ? |
164 | T7 | *(engine)* `GET /api/capabilities` + `GET /api/health` + banner | T1 | coord | ?
| ? |
165 | T8 | *(engine)* `GET /api/videos` + `GET /api/reels/{video_id}` (S4 reload-survival) |
? | coord | ? | ? |
166 | T9 | S5 Settings (config+capabilities; disabled compute/storage target) | T7,T8 | no |
? | ? |
167 | T10 | Observability pass: correlation id end-to-end + structured logs + metrics ($5) |
T4 | no | ? | ? |
168 | T11 | Retention sweep for `output/`+`uploads/` | ? | owner | ? | ? |
169 | T12 | Ingest from **bucket URI** (`s3://`/`gs://`) wiring ? **engine half DONE**
(`/api/process` accepts URIs, M2/[#280], `[verified]`); remaining = the SPA ingest call (=
**B-P2**) | T4 | coord | ? | engine done; SPA pending |
170 | T13 | **GPU host stand-up** (provider-agnostic) + wire `detector_impl=native`; owner
byte-parity (P4) | branch | ? owner | ? | ? |
171 | T14 | *(reserved)* Bucket-worker bridge ? remote-GPU "compute as a setting" (P5) |
T8,T12 | ? owner | ? | ? |
172
173 **Owner priority (2026-06-20): T13 (GPU host) is elevated to run in parallel from the
start ? owner provisions the host + token; the engine session lands native WASB; the UI MVP
does **not** block on it (ships on Gemini).
174
175 ## 10. Open questions ? owner / external
176 **Blocking gates:** (1) **edge-auth** ? Caddy basicauth (self-contained) vs
cloudflared+Cloudflare Access (matches the two-host design, no inbound ports)? (2) **retention
defaults** ? auto-delete originals after compile? keep proxies+intervals only? window?
(privacy/consent). (3) **per-video Gemini/GPU cost ceiling** ? what cap, hard-fail vs warn? (4)
**GPU host choice** ? DO GPU droplet / GCE GPU VM / Cloud Run+GPU (and is a containerized worker
worth building for Cloud Run)? (5) **P5 bridge ownership** ? who builds the orchestration
endpoint + how it coordinates with the active worker sessions.
177 **Nice-to-knows:** console home (web/ vs engine static); a review-playback streaming
endpoint; per-user scoping timing; containerization vs runbook.
178
179 ## 11. Definition of done
180 - [ ] CUJ-1 works end to end on the droplet (Gemini **and** stub) behind the gate, via
the ingest menu.
181 - [ ] Raw `:8000`/`:900x` unreachable from outside (**tested**, not assumed).
182 - [ ] Local upload wired; bucket-URI ingest at least designed/tracked; Drive scoped
honestly (not faked).
183 - [ ] RallyPicker embedded with honesty rules; `min_shot_count` surfaced.
184 - [ ] `api/capabilities` + `api/health` + banner truthful; reload-survival via
`api/videos` + `api/reels`.
185 - [ ] **Observability:** correlation id threaded end-to-end; structured logs at every
new path; loud failures; logs inspected during CUJ replay.
186 - [ ] Retention sweep in place; `GEMINI_API_KEY` rotated; `allowed_video_root` set.
187 - [ ] CI green (L2 contract + L4 regression); coverage ? floor; $9 tracker all
?/?-resolved.
188
189 ## 12. References
190 **Engine code `[verified this session]`:**
`backend/main.py:12-17,42,52,307-328,331-340,342-396,638`;
`backend/api/uploads.py:64,89,123,147`; `backend/api/job_worker.py:37`;
`backend/api/static/app.js:88,344,373`;
`backend/storage/{base.py,local.py:33,ref.py,__init__.py:43}`;
`backend/config/models.py:129,183,298`;
`backend/pipeline/segmenters/{trajectory_hybrid.py:57-63,76-85, fusion_hybrid.py:76-86,
gemini.py:470-485, motion.py:191-198}`; `backend/pipeline/detectors/{native_wasb_runner.py,
stub_runner.py}`; `backend/worker/__main__.py:88,98,149`; `config.json` (`default_segmenter:
gemini`). **Engine docs:** `HOSTING_PLAN.md:42,102-108`; `VIDEO_LOCALITY_MODEL.md`;
`DESKTOP_APP_PLAN.md`; `SETUP_NEW_MACHINE.md`; `PLATFORM_ARCHITECTURE.md`;
`DECOUPLED_COMPUTE/{README,02,04,07,HANDOFF}.md`; `UI_PROPOSAL.md:41-66`;
`PROJECT_DOC_TEMPLATE.md`. **This repo:** `PER_RALLY_SELECTION.md`, `REQUEST_ACCESS_FORM.md`,

```

```
`PORTABILITY.md`, `web/README.md`.
```

191

192 ---

193 ### Changelog

194 - 2026-06-20 ? Initial `DRAFT`. Consolidated three single-box-appliance designs (FastAPI-native / unified-origin / decoupled-profile) into the reverse-proxied unified-origin spine + compute-as-a-setting framing, via a multi-agent design pass; every engine-fact anchor re-verified against source before writing (catching the native-WASB partial-wiring + `ai\_max\_workers` default nuances). Folded in the owner's ingest-menu (Drive/bucket/local), multi-provider GPU, and first-class Observability directives.

195

11. user (2026-06-21T13:52:50.231Z)

Web search results for query: "Ollama Windows installer how it distributes models GPU CUDA bundled update mechanism"

Links: [{"title": "Windows - Ollama", "url": "https://docs.ollama.com/windows"}, {"title": "Ollama Windows FAQ ? Install, Models, GPU, Offline & Troubleshooting", "url": "https://ai-ollama.github.io/faq.html"}, {"title": "How to Install Ollama on Windows ? Official Step-by-Step Guide 2026", "url": "https://ai-ollama.github.io/install-windows.html"}, {"title": "How to Install Ollama on Windows (2026): Step-by-Step | Local AI Master", "url": "https://localaimaster.com/blog/ollama-windows-installation"}, {"title": "How to Install Ollama with NVIDIA GPU Support (CUDA Guide) | Markaicode", "url": "https://markaicode.com/install-ollama-nvidia-gpu-cuda-support/"}, {"title": "Install Ollama under Win11 & WSL - CUDA Installation guide · GitHub", "url": "https://gist.github.com/nekiee13/c8ec43bce5fd75d20e38b31a613fd83d"}, {"title": "Releases · ollama/ollama", "url": "https://github.com/ollama/ollama/releases"}, {"title": "Download Ollama for Windows ? Official Installer & Setup Guide 2026", "url": "https://ai-ollama.github.io/"}]

Based on the search results, here's what I found about Ollama's Windows installer, GPU support, and model distribution:

Windows Installer

The easiest way to install Ollama on Windows is to use the OllamaSetup.exe installer, which is ~5 MB. It installs in your account without requiring Administrator rights, and a short wizard installs Ollama and starts the service automatically.

Model Distribution

The first pull downloads the model weights (~4?8 GB depending on the model), and after that it runs fully offline. Models are stored in C:\Users\YourName\.ollama\models.

GPU/CUDA Support

Ollama automatically detects NVIDIA GPUs via CUDA and AMD GPUs via DirectML on Windows. For NVIDIA specifically, you need to update to the latest Game Ready or Studio driver (version 527+) and ensure CUDA is installed. Ollama automatically uses your GPU when CUDA is available.

Update Mechanism

To update, quit Ollama from the system tray, download the new installer, and run it over the existing installation?models are preserved. Downloaded updates are stored in explorer %LOCALAPPDATA%\ollama.

The installer does not bundle models by default; instead, users download them on-demand using the `ollama pull` command, allowing for flexible, incremental model acquisition.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

12. user (2026-06-21T13:52:53.808Z)

Web search results for query: "AUTOMATIC1111 stable-diffusion-webui Windows one-click installer python venv torch install UX"

Links: [{"title": "One Click Installer · AUTOMATIC1111/stable-diffusion-webui · Discussion #8020", "url": "https://github.com/AUTOMATIC1111/stable-diffusion-webui/discussions/8020"}, {"title": "How to install Stable Diffusion on Windows (AUTOMATIC1111) - Stable Diffusion Art", "url": "https://stable-diffusion-art.com/install-windows/"}, {"title": "GitHub - AUTOMATIC1111/stable-diffusion-webui: Stable Diffusion web UI · GitHub", "url": "https://github.com/AUTOMATIC1111/stable-diffusion-webui"}, {"title": "How to Install Stable Diffusion with AUTOMATIC1111 UI on Windows ? LeoTheLegion", "url": "https://leothelegion.net/2025/02/17/how-to-install-stable-diffusion-with-ui-on-windows/"}, {"title": "AUTOMATIC1111 SDUI One-Click Install + Optimization | Windows Guide - YouTube", "url": "https://www.youtube.com/watch?v=HK3pluCh9Cs"}, {"title": "How to run Stable Diffusion XL on your own computer, with one click. | by cocktail peanut | Medium", "url": "https://medium.com/@cocktailpeanut/how-to-run-stable-diffusion-xl-on-your-own-computer-with-one-click-4c193813cb06"}, {"title": "Install and Update Automatic1111 on Windows/Mac/Linux", "url": "https://www.stablediffusiontutorials.com/2023/12/install-run-automatic1111.html"}, {"title": "Install AUTOMATIC1111's Stable Diffusion WebUI on Windows (NVIDIA GPU) - Aituts", "url": "https://aituts.com/stable-diffusion-on-windows-automatic1111/"}, {"title": "How to Install Stable Diffusion Hassle-Free on Your Computer - Next Diffusion | Next Diffusion", "url": "https://www.nextdiffusion.ai/tutorials/how-to-install-stable-diffusion-web-ui"}]

Search Results: AUTOMATIC1111 stable-diffusion-webui Windows Installation

I've found comprehensive information about the AUTOMATIC1111 stable-diffusion-webui Windows setup. Here are the key findings:

Windows Installation Overview

AUTOMATIC1111 requires Python 3.10.6 installation, as newer versions of Python do not support torch. The installation involves installing git, downloading the stable-diffusion-webui repository by running git clone, and then running webui-user.bat from Windows Explorer as a normal non-administrator user.

One-Click Installer Options

Users have noted that the official AUTOMATIC1111 Windows one-click installer does not install Python and git automatically, whereas other third-party installers like CMDR's do. Additionally, alternative one-click launchers provide installation with no terminals and no complicated steps, where just a click is needed and it works.

Python Virtual Environment (venv) and PyTorch (Torch)

The official installation process automatically handles dependency installation. If issues arise, users can delete the venv and tmp folders to reset the environment. Running webui-user.bat takes several minutes for the Web UI to install required modules and download the Stable Diffusion model.

User Experience

The initial setup takes approximately 5-10 minutes, but subsequent launches start instantly. After setup completes, users access the interface by opening <http://localhost:7860> in their web browser.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

13. assistant (2026-06-21T13:52:57.363Z)

I have strong grounding on our app and the appliance design. Let me research the remaining apps

and the closest analogs (Frigate's app-store distribution, ComfyUI desktop installer internals, Pinokio, LM Studio, Jan, whisper GUIs).

14. user (2026-06-21T13:53:06.660Z)

Web search results for query: "Frigate Home Assistant add-on distribution; how Frigate ships ffmpeg inside Docker image versions"

Links: [{"title": "Installation | Frigate", "url": "https://docs.frigate.video/frigate/installation/"}, {"title": "Advanced Options | Frigate", "url": "https://docs.frigate.video/configuration/advanced/"}, {"title": "Frigate FFMPEG issues on hw acceleration - Third party integrations - Home Assistant Community", "url": "https://community.home-assistant.io/t/frigate-ffmpeg-issues-on-hw-acceleration/712826"}, {"title": "Frigate NVR with Docker and Home Assistant", "url": "https://www.mostlychris.com/frigate-nvr-with-docker-and-home-assistant/"}, {"title": "Getting started | Frigate", "url": "https://docs.frigate.video/guides/getting_started/"}, {"title": "Frigate Configuration | Frigate", "url": "https://docs.frigate.video/configuration/"}, {"title": "Anyone has success running Frigate in Home Assistant? | IP Cam Talk", "url": "https://ipcamtalk.com/threads/anyone-has-success-running-frigate-in-home-assistant.75730/"}, {"title": "Releases · blakeblackshear/frigate", "url": "https://github.com/blakeblackshear/frigate/releases"}, {"title": "How to Set Up Frigate NVR with Home Assistant: Complete Guide | SmartWired", "url": "https://smart-wired.com/cameras/frigate-nvr-setup/"}, {"title": "[Support]: ffmpeg missing from Docker container · blakeblackshear/frigate · Discussion #16892", "url": "https://github.com/blakeblackshear/frigate/discussions/16892"}]

Based on the search results, here's what I found about Frigate's ffmpeg distribution in Docker images:

ffmpeg Bundling in Frigate Docker Images

Frigate includes ffmpeg, with version 7 being included in recent releases. The way ffmpeg is shipped within Docker images has changed over time, particularly with significant updates to the project.

Location of ffmpeg Binary

The container provides ffmpeg binaries in /usr/lib/ffmpeg. This represents a change from earlier versions where ffmpeg was accessed differently.

Breaking Changes in Recent Versions

Frigate will no longer fall back to look for a ffmpeg binary in PATH. If you are extending the Frigate image or somehow injecting a custom ffmpeg binary in a place like /usr/local/bin, please read the custom ffmpeg build guide.

Custom ffmpeg Option

For users needing a different ffmpeg version, there's an alternative approach: Statically built ffmpeg and ffprobe binaries can be placed in /config/custom-ffmpeg/bin for Frigate to use, and users should download their ffmpeg build and uncompress it to the /config/custom-ffmpeg folder, then verify that both the ffmpeg and ffprobe binaries are located in /config/custom-ffmpeg/bin.

Home Assistant Add-on Considerations

Home Assistant Add-on users need to be aware of several important breaking changes, as detailed in the Important Notes for Home Assistant Add-on Users section. This suggests that the ffmpeg distribution method may have special considerations when running Frigate as a Home Assistant add-on.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

15. user (2026-06-21T13:53:07.641Z)

Web search results for query: "LM Studio architecture local server llama.cpp bundled runtime model download install windows update"

Links: [{"title": "Local LLM Guide: Ollama, LM Studio & llama.cpp in 2026 | Claude 5 Hub", "url": "https://claude5.com/news/local-llm-guide-ollama-lm-studio-llama-cpp-in-2026"}, {"title": "Self-host LLMs in production with llama.cpp llama-server", "url": "https://docs.servicestack.net/ai-server/llama-server"}, {"title": "Welcome to LM Studio Docs! | LM Studio", "url": "https://lmstudio.ai/docs/app"}, {"title": "How to Run LLMs Locally with LM Studio: Complete Guide 2026 | by Kapustin Mykhailo | Medium", "url": "https://medium.com/@kapustinomm/how-to-run-llms-locally-with-lm-studio-complete-guide-2026-c941843e000e"}, {"title": "LM Studio - Local AI on your computer", "url": "https://lmstudio.ai/"}, {"title": "I Switched From Ollama And LM Studio To llama.cpp And Absolutely Loving It", "url": "https://itsfoss.com/llama-cpp/"}, {"title": "Llama.cpp vs LM Studio - Which Local LLM Tool is Better?", "url": "https://llama-cpp.com/llama-cpp-vs-lm-studio/"}, {"title": "How to run LLMs on PC at home using Llama.cpp", "url": "https://www.theregister.com/2025/08/24/llama_cpp_hands_on/"}, {"title": "Introducing LM Studio 0.4.0 | LM Studio Blog | LM Studio", "url": "https://lmstudio.ai/blog/0.4.0"}, {"title": "Pairing Claude Code with Local Models - KDnuggets", "url": "https://www.kdnuggets.com/pairing-claude-code-with-local-models"}]

Based on the search results, here's comprehensive information about LM Studio's architecture, local server capabilities, llama.cpp integration, bundled runtime, and Windows setup:

Architecture

LM Studio v0.4+ features a headless daemon called `llmster` that decouples the GUI from inference through an HTTP API, allowing the daemon to run in the background on a server or CI pipeline. LM Studio's stack consists of three layers: the User Interface (Desktop GUI, CLI), the API & SDK layer (OpenAI-compatible REST endpoints), and the Inference Engine layer (llama.cpp, Apple MLX, GPU offloading).

Inference Engine & llama.cpp

LM Studio supports running LLMs on Mac, Windows, and Linux using llama.cpp. llama.cpp is the engine under many local LLM tools and is a C++ implementation optimized for CPU and GPU inference with advanced quantization that dramatically reduces memory footprint.

Bundled Runtime

LM Studio's llama.cpp engine graduated to version 2.0.0, introducing support for concurrent inference requests to the same model. To install or manage LM Runtimes, press Ctrl Shift R on Windows/Linux.

Model Download & Installation

LM Studio provides a graphical interface for discovering and downloading models, with a built-in model hub that lets you browse Hugging Face and chat with models directly in the app. To get LM Studio, download an installer from the Downloads page?it's available for macOS, Windows, and Linux.

Windows Setup & Server Capabilities

LM Studio exposes a local API server compatible with OpenAI's client libraries, making it easy to swap into existing workflows. LM Studio can operate entirely offline once you download model files, and it can run without the GUI for server deployments.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

16. user (2026-06-21T13:53:07.896Z)

Web search results for query: "Jan AI Tauri app local server bundled engine model download offline; GPT4All installer distribution"

Links: [{"title": "GitHub - janhq/jan: Jan is an open source alternative to ChatGPT that runs 100% offline on your computer. · GitHub", "url": "https://github.com/janhq/jan"}, {"title": "Jan: The Offline AI Assistant Every Developer Needs", "url": "https://prompts.brightcoding.dev/blog/jan-the-revolutionary-offline-ai-assistant-every-developer-needs"}, {"title": "Local API Server", "url": "https://www.jan.ai/docs/desktop/api-server"}, {"title": "Jan AI - Download and install on Windows | Microsoft Store", "url": "https://apps.microsoft.com/detail/xpdcnfn5cpzlbq?hl=en-US&gl=US"}, {"title": "Jan - Open-Source ChatGPT Replacement", "url": "https://www.jan.ai/"}, {"title": "Jan.ai download | SourceForge.net", "url": "https://sourceforge.net/projects/jan.mirror/"}, {"title": "Releases · janhq/jan", "url": "https://github.com/janhq/jan/releases"}, {"title": "How to Use Jan.ai: Run Private AI Models Locally", "url": "https://roundproxies.com/blog/jan-ai/"}, {"title": "Jan AI: The Open-Source Local LLM Desktop App Explained - ML Journey", "url": "https://mljourney.com/jan-ai-the-open-source-local-llm-desktop-app-explained/"}, {"title": "How to run AI models locally as a beginner?", "url": "https://www.jan.ai/post/run-ai-models-locally"}]

Links: [{"title": "Quickstart - GPT4All Documentation", "url": "https://docs.gpt4all.io/gpt4all_desktop/quickstart.html"}, {"title": "How to Run GPT4All Models Locally 2025: Complete Guide", "url": "https://www.propelrc.com/how-to-run-gpt4all-models-locally/"}, {"title": "GitHub - nomic-ai/gpt4all: GPT4All: Run Local LLMs on Any Device. Open-source and available for commercial use. · GitHub", "url": "https://github.com/nomic-ai/gpt4all"}, {"title": "How to Run LLMs Model Locally - GeeksforGeeks", "url": "https://www.geeksforgeeks.org/blogs/how-to-run-llms-model-locally/"}, {"title": "Running GPT4All LLM Locally, No Internet Needed ? Just a Few Lines of Code! | by Anand M | Medium", "url": "https://medium.com/@anandmali/running-gpt4all-llm-locally-no-internet-needed-just-a-few-lines-of-code-e134cccf6974"}, {"title": "Install and Run GPT4All Locally with Secure API Access", "url": "https://www.centron.de/en/tutorial/install-and-run-gpt4all-locally-with-secure-api-access/"}, {"title": "GPT4All Setup: Install and Environment Variables That Actually Stick | Markaicode", "url": "https://markaicode.com/howto/gpt4all-install-and-environment-variables-setup/"}, {"title": "Models - GPT4All Documentation", "url": "https://docs.gpt4all.io/gpt4all_desktop/models.html"}, {"title": "How to Install GPT4ALL: Open Source LLM Chatbot Guide | Vultr Docs", "url": "https://docs.vultr.com/how-to-install-gpt4all-an-opensource-chatbot-application-for-running-large-language-models-llms"}]

Summary of Search Results

I've found comprehensive information about both Jan AI and GPT4All. Here's what the searches revealed:

Jan AI (Tauri-based Desktop Application)

Jan leverages Tauri for its desktop framework, and the application bundles llama.cpp as its inference engine, providing optimized CPU and GPU acceleration across platforms.

****Local Server & Offline Capabilities:****

- Jan includes an OpenAI-compatible API at localhost:1337 for other applications
- The server is ready when the logs show JAN API listening at http://127.0.0.1:1337
- Users can download and run LLMs with full control and privacy

****Model Management:****

- Users can download and run LLMs (Llama, Gemma, Qwen, GPT-oss etc.) from HuggingFace
- The installer bundles everything including llama.cpp for inference, so no separate backend is needed

GPT4All (Desktop Application with API Server)

****Installation & Distribution:****

- Download the GPT4All installer for your operating system, run it with administrator privileges, and follow the setup wizard ? total time is 10-30 minutes
- Select the appropriate version for your operating system (Mac, Windows or Linux)

****Local Model Support:****

- LLMs are downloaded to your device so you can run them locally and privately
- GPT4All connects you with LLMs from HuggingFace with a llama.cpp backend so that they will run efficiently on your hardware
- Many of these models can be identified by the file type .gguf

****API Server:****

- Use the provided API endpoint (e.g., <http://localhost:4891>) to interact with the model programmatically

Both applications support offline operation with bundled inference engines and provide local API servers for integration with other tools.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

17. user (2026-06-21T13:53:08.963Z)

Web search results for query: "Pinokio AI app installer one-click bundled python conda venv ffmpeg manages environments"

Links: [{"title": "Pinokio: One-Click Launcher for Open-Source AI Models, Local Apps & Scripts (Windows, macOS, Linux) - Firethering", "url": "https://firethering.com/pinokio-one-click-launcher-for-open-source-ai-llm-local-apps/"}, {"title": "GitHub - pinokiocomputer/pinokio: AI Browser · GitHub", "url": "https://github.com/pinokiocomputer/pinokio"}, {"title": "Pinokio - Download - Softpedia", "url": "https://www.softpedia.com/get/System/Pinokio.shtml"}, {"title": "Releases · pino kiocomputer/pinokio", "url": "https://github.com/pinokiocomputer/pinokio/releases"}, {"title": "GitHub - 6Morpheus6/pinokio-wiki: Tutorial for Pinokio and its Applications · GitHub", "url": "https://github.com/6Morpheus6/pinokio-wiki"}, {"title": "Pinokio Manual", "url": "https://desktop.pinokio.co/docs/"}, {"title": "On-click Install Local AI Applications Using Pinokio | Hackaday", "url": "https://hackaday.com/2024/02/26/on-click-install-local-ai-applications-using-pinokio/"}, {"title": "Pinokio", "url": "https://beta.pinokio.co/"}, {"title": "Pinokio makes installing ComfyUI a breeze", "url": "https://www.xda-developers.com/pinokio-how-to/"}]

Based on the search results, here's comprehensive information about Pinokio AI app installer:

Overview

Pinokio is an application that helps you install and run complex programs, especially AI tools, on your computer with just one click. Pinokio is a 1-click launcher for any open-source project. Think of it as a terminal application with a user-friendly interface that can programmatically interact with scripts.

One-Click Installation with Bundled Components

With one click, the app automatically downloads the app, sets up Python, Conda, virtual environments, GPU libraries and all required dependencies, then launches a local web interface for you in your browser. Pinokio installs the latest version of FFMPEG to support new features, including torchcodec APIs.

Environment Management

By default the entire thing is run isolated in the pinokio activated conda environment, and the execution path is the downloaded app's path. The python dependencies are also installed in an app-isolated manner. The built-in package managers (Conda, Homebrew, Pip, and NPM) install everything inside the isolated pinokio home path (~/.pinokio) by default.

A declarative syntax for automatically creating or activating a venv environment at the specified path. When specified: Creates a venv at the specified path if it doesn't exist yet, or if it exists, activates the existing venv at the specified path, and runs the shell session in that venv.

Out-of-the-Box Tools

All-in-one environment ready to run ANY web app/AI servers on your PC, including Python, Node.js, Bun, Git, tmux (macOS/Linux), Conda, and more out of the box.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

18. assistant (2026-06-21T13:53:16.362Z)

Excellent. Pinokio is a very strong analog (handles the exact dual-conda-env + ffmpeg + GPU problem we have). Let me dig into the ComfyUI desktop installer internals (it's the closest "Python+torch+web UI packaged for non-tech users" precedent), Frigate's update mechanism, and the whisper GUI patterns. Let me also verify our worker/server entrypoint and the dual-env runner.

19. user (2026-06-21T13:53:23.031Z)

ComfyUI Desktop App Architecture

Distribution & Installation

The app uses **NSIS installer** for Windows and **DMG** for macOS, packaged with Electron. Files install to `%LOCALAPPDATA%\Programs\ComfyUI` (Windows) or `~/Library/Application Support/ComfyUI` (macOS).

Python & Dependencies Management

The desktop app bundles **uv** (a Python package manager) rather than embedding Python directly. As stated in the repository: "On startup, it will install all the necessary python dependencies with uv and start the ComfyUI server."

The app includes compiled Python requirements stored in `.compiled`` files, with documentation in header comments explaining how they're produced.

Core Components

The application bundles:

- ComfyUI source code (from official releases)
- ComfyUI-Manager (installed via pip when enabled)
- Electron and Chromium binaries
- Node modules

Updates & Model Management

"The app will automatically update with stable releases of ComfyUI, ComfyUI-Manager (pip), and the uv executable." Users select a storage location for models, inputs, outputs, and workflows, configured in `config.json`` and `extra_models_config.yaml``.

License

The project is licensed under **GPL-3.0**.

Development

Built with TypeScript (89.9%), using Vite for bundling and Todesktop for building/codesigning distributables.

20. user (2026-06-21T13:53:26.030Z)

Web search results for query: "Fooocus stable diffusion one-click package embedded python pytorch download Windows beginner no install"

Links: [{"title": "The Stable Diffusion Fooocus Guide: Getting started", "url": "https://andreaskuhr.com/en/the-stable-diffusion-fooocus-guide-getting-started.html"}, {"title": "GitHub - llyasviel/Fooocus: Focus on prompting and generating ."}]

```
GitHub", "url": "https://github.com/lllyasviel/Fooocus"}, {"title": "Fooocus Download & Install Guide (2025) ? How to Run It Step-by-Step", "url": "https://techyheaven.com/fooocus-download-install-guide/"}, {"title": "One Click Install and run Fooocus on Colab/PC", "url": "https://www.stablediffusiontutorials.com/2023/12/install-run-fooocus-on-pc-colab.html"}, {"title": "Download Cuda 12.1 + Pytorch 2.31_ Copy of Forge one-click Install package (Sept 2024) source code.zip (stable-diffusion-webui-forge)", "url": "https://sourceforge.net/projects/stable-diffusion-webui-forge/files/latest_cu121/Cuda%2012.1%20%2B%20Pytorch%202.31_%20Copy%20of%20Forge%20one-click%20Install%20package%20%28Sept%202024%29%20source%20code.zip/download"}, {"title": "Fooocus Guide 2026: Easiest Stable Diffusion UI | Local AI Master", "url": "https://localaimaster.com/blog/fooocus-guide"}, {"title": "Fooocus - Best Stable Diffusion UI for Prompt Engineering", "url": "http://anakin.ai/blog/fooocus-stable-diffusion/"}, {"title": "Fooocus: Stable Diffusion simplified - Stable Diffusion Art", "url": "https://stable-diffusion-art.com/fooocus/"}, {"title": "Fooocus: Hassle-free Stable Diffusion experience", "url": "https://dataconomy.com/2023/11/30/fooocus-features-and-how-to-install/"}]
```

Based on the search results, here's comprehensive information about Fooocus as a one-click Stable Diffusion package for Windows:

What is Fooocus?

Fooocus is an open-source image generation software that simplifies the process of creating images from text prompts, built on Gradio and leveraging Stable Diffusion XL, eliminating the need for manual parameter tweaking.

Installation on Windows

The Windows installation is straightforward: extract the zip file to your desired folder, double-click run.bat to start Fooocus, and it will download models the first time you run. Python is embedded, requiring no installation.

Key Features for Beginners

Fooocus is the ideal user interface for beginners and Midjourney users who want to generate images on their own computer independently of providers. It features opinionated SDXL defaults, automatic prompt expansion, curated style presets, and one-click "Speed / Quality / Extreme Speed" performance modes.

Unlike other SD interfaces like ComfyUI, Fooocus requires just a single prompt, making it easy for even beginners to get started.

System Requirements

Requirements include Windows 10 or 11, 30 GB free hard drive space, 8 GB RAM memory, and at least a separate NVIDIA graphics card with 4 GB VRAM (6 GB recommended).

The search results confirm Fooocus is indeed designed as a beginner-friendly, one-click package with embedded Python and PyTorch for Windows users.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

21. user (2026-06-21T13:53:26.969Z)

Web search results for query: "whisperX faster-whisper GUI Windows app bundled cuDNN cublas torch packaging problems Buzz Subtitle Edit"

Links: [{"title": "Get Raw Subtitles for Any Video or Audio with Faster Whisper - WellWells", "url": "https://welltsai.com/en/post/whisper/"}, {"title": "Faster-Whisper Setup Guide (2026): 4x Faster Local Speech-to-Text | Local AI Master", "url": "https://localaimaster.com/blog/faster-whisper-guide"}, {"title": "How I use

whisper-faster on my machine - VideoHelp
Forum", "url": "https://forum.videohelp.com/threads/410865-How-I-use-whisper-faster-on-my-machine"}, {"title": "GitHub - SYSTRAN/faster-whisper: Faster Whisper transcription with CTranslate2 · GitHub", "url": "https://github.com/SYSTRAN/faster-whisper"}, {"title": "GitHub - tsmdt/whisply: ? Fast, cross-platform CLI and GUI for batch transcription, translation, speaker annotation and subtitle generation using OpenAI's Whisper on CPU, Nvidia GPU and Apple MLX.", "url": "https://github.com/tsmdt/whisply"}, {"title": "GitHub - m-bain/whisperX: WhisperX: Automatic Speech Recognition with Word-level Timestamps (& Diarization) · GitHub", "url": "https://github.com/m-bain/whisperx"}, {"title": "GitHub - chboishabba/WhisperX-WebUI: A Web UI for easy subtitle using whisperx model based on whisper-webui by jhj0517. · GitHub", "url": "https://github.com/chboishabba/WhisperX-WebUI"}, {"title": "Voice recognition with Whisper in SubtitleEdit - VideoHelp Forum", "url": "https://forum.videohelp.com/threads/419047-Voice-recognition-with-Whisper-in-SubtitleEdit"}, {"title": "Releases · Purfview/whisper-standalone-win", "url": "https://github.com/Purfview/whisper-standalone-win/releases"}]]

Based on the search results, I found relevant information about whisperX, faster-whisper, GUI Windows apps, and their packaging issues with cuDNN, cuBLAS, and torch.

Key Findings:

****WhisperX and Faster-Whisper Overview:****

WhisperX builds on faster-whisper by adding advanced features like batch inference, forced alignment, speaker diarization, and voice activity detection (VAD), and is particularly useful for precise transcription and analysis of long audio files or recordings with multiple speakers.

****GUI Windows Applications:****

Several GUI applications bundle these tools for Windows:

- whisper-standalone-win provides standalone CLI executables of faster-whisper for Windows, Linux & macOS
- whisply combines faster-whisper and mlx-whisper to offer an easy-to-use solution for batch processing files on Windows, Linux and Mac, and also enables word-level speaker annotation by integrating whisperX and pyannote
- Whisper-WebUI bundles whisperX so you can generate accurate word-level timestamps and automatically assign speaker labels in a single pass, with Python dependencies installed automatically when you run `pip install -r requirements.txt`

****Packaging Issues with cuDNN and cuBLAS:****

The latest versions of ctranslate2 only support CUDA 12 and cuDNN 9, and for CUDA 11 and cuDNN 8, the current workaround is downgrading to the 3.24.0 version of ctranslate2

One solution involves downloading cuBLAS and cuDNN libraries and extracting them to the same folder as the whisper-faster executable

****Subtitle Edit Integration:****

Subtitle Edit offers different Whisper engines including OpenAI, Purfview's Faster-Whisper-XXL, CPP, CPP cuBLAS, const-me, stable-ts, and WhisperX

The search results don't specifically mention a "Buzz Subtitle Edit" application, but they do cover the broader ecosystem of whisper-based transcription tools and their Windows packaging challenges.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

22. assistant (2026-06-21T13:53:31.601Z)

Now I have rich data. Note ComfyUI desktop is GPL-3.0 (license-relevant flag for us). Let me verify our engine's actual entrypoints and dual-env runner, plus check the prior DESKTOP_APP_PLAN.md for what was already considered, and do one more search on Frigate's

update/distribution channels and the Fooocus embedded-python+torch download pattern detail.

23. user (2026-06-21T13:53:33.036Z)

```
1      # Cross-Platform Desktop App ? "Plug in your card, get your highlights"
2
3      > **Status:** `DRAFT` ? `owner-gated` (scoping doc only; **nothing is built**) .
**Owner:** avidullu . **Created:** 2026-06-18 . **Last updated:** 2026-06-18
4      > **Lifecycle:** `DRAFT` ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5      > **Tracking anchors:** §7 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in memory `current-state` (and `docs/NEXT_STEPS.md` once the owner
greenlights P0).
6      > **Relation to existing docs:** **peer-of**
[PLATFORM_ARCHITECTURE.md](PLATFORM_ARCHITECTURE.md) (this is the **`LocalDesktop`
ComputeBackend** made into a product) . **realizes the L0 tier** of
[VIDEO_LOCALITY_MODEL.md](VIDEO_LOCALITY_MODEL.md) . **consumes**
[OWNED_MODEL_IMPLEMENTATION_PLAN.md](OWNED_MODEL_IMPLEMENTATION_PLAN.md) (the smaller/export-
clean model is one answer to the compute crux) . bound by
[COMMERCIALIZATION.md](COMMERCIALIZATION.md) licensing guardrails.
7      > **Honesty note:** every claim is tagged `[verified]` (checked against code/measured),
`[researched]` (needs sign-off), or `[design]` (proposed). **The entire build is `[design]`** ?
this doc scopes; it does not ship. Not legal advice.
8
9      ---
10
11     ## 0. TL;DR
12     Package the mature local pipeline (WASB shuttle detector + improved windowing + ffmpeg
stitch, already runnable fully-offline via [tools/generate_highlights.py --skip-ai-
handoff](../tools/generate_highlights.py)) as a **cross-platform desktop app** for **non-
technical recreational players**: plug in the camera's USB/SD card, click once, get back **only
the highlight reels** ? fully on-device, **no upload**, **no copy of the GBs of 4K originals**.
This is the product surface that fits the "local, cheap, efficient" thesis and the L0 ("nothing
leaves the machine") tier of `VIDEO_LOCALITY_MODEL.md`.
13
14     **The make-or-break unknown is compute on a typical enthusiast laptop** ? which often
has **no discrete GPU**. WASB runs at **~3.4x real-time on a laptop RTX 2070 Super** `[verified,
measured]`; on CPU it is far slower, and Apple Silicon has **no CUDA** at all. **P0 is a
feasibility spike that measures CPU and Apple-MPS throughput for a ~10-min clip before any app
is built.** If CPU-only is unacceptable on commodity hardware, the path needs a smaller/faster
owned model (ties `OWNED_MODEL_IMPLEMENTATION_PLAN`) or a tiered "fast-if-GPU, slow-but-works-
if-CPU" design. **Everything past P0 is gated on that result.**
15
16     ## 1. Problem & goal
17
18     **Why now / the user.** Target users are **recreational sports enthusiasts, not tech-
savvy**. Their match videos are **huge** (GBs of 4K). The cloud-upload product surface fails
them on every axis: slow on a typical Indian uplink (`VIDEO_LOCALITY_MODEL.md` §0: 10 GB ? ~1 h
at 20 Mbps, ~4.5 h at 5 Mbps), bandwidth-costly, privacy-fraught, and it bloats both our storage
and their drives. The thing the customer actually wants back is **kilobytes of timestamps ? a
highlight reel** (`VIDEO_LOCALITY_MODEL.md` §1: "timestamps are the product; the video is dead
weight").
19
20     **The concrete outcome ("good" looks like this):**
21     1. User plugs the GoPro/phone SD card or camera USB into their laptop.
22     2. The app detects the card, lists the match videos on it.
23     3. User clicks **Generate Highlights**.
24     4. The app makes an on-device proxy, runs WASB detection + windowing + stitches the reel
? **100% locally**.
25     5. The reel(s) land in a user-chosen folder. **The originals are never copied off the
card and never bloat any drive.**
26
27     No account, no upload, no cloud dependency in the default path. Gemini stays an
**optional** "better brain if you want it" toggle ? never required, never bundled as a hard
dependency, never a training source (`COMMERCIALIZATION` C4).
```

```

28
29      **Read to get current:** `VIDEO_LOCALITY_MODEL.md` (the L0/locality framing + Q05
packaging seed), `PLATFORM_ARCHITECTURE.md` §3.2 (the ComputeBackend seam),
[`backend/pipeline/detectors/base.py`](../backend/pipeline/detectors/base.py) (the detector ABC
that already anticipates a Linux-native runner), `tools/generate_highlights.py` (the local
entrypoint this app wraps).
30
31      ## 2. Decisions locked
32
33      | # | Decision | Source / date | Implication |
34      |---|-----|-----|-----|
35      | D1 | **Default path is 100% on-device, no upload, no Gemini.** The reel is produced
from the local pipeline (`--skip-ai-handoff`). | This doc · 2026-06-18; matches
`VIDEO_LOCALITY_MODEL` L0 | The app must never *require* a network call. Gemini is an opt-in
toggle only. |
36      | D2 | **Never copy the originals.** The app reads source video in place (off the card /
a chosen folder) and writes **only** reels + a tiny report to a user folder. | This doc; mirrors
`generate_highlights.py` non-destructive `_atomic_publish` discipline | No "import library" that
duplicates 10 GB files. Detection runs on a proxy; stitch reads the original in place. |
37      | D3 | **Drop the WSL2 dependency; run the detector natively per-OS.** | This doc;
enabled by the `DetectorRunner` ABC `[verified]` | Windows-native (no WSL), Linux+NVIDIA (CUDA
directly), macOS (MPS or CPU). The WSL adapter stays for the current dev box but is **not** the
shipped path. |
38      | D4 | **Commercial-clean licensing for everything bundled: MIT / Apache-2.0 / BSD
only** (code, weights). ffmpeg ships as an **LGPL build** (no GPL-only encoders like x264). |
`COMMERCIALIZATION` (ratified 2026-06-09); `VIDEO_LOCALITY_MODEL` Q05 (**ffmpeg LGPL build =
licensing-clean**) | WASB-SBDT weights are MIT `[verified]` (see `tracknet-wasb-wsl2-decision`).
The stitch must use openh264 (BSD) or OS hardware encoders, **not** statically-linked x264. See
§3.4. |
39      | D5 | **Reuse the existing pipeline verbatim; add only the shell + a native runner +
device-I/O.** | `backend/pipeline/detectors/base.py` reuse contract `[verified]` | No re-
architecture of windowing/stitch. New code = native runner, USB/SD detection, app shell,
packaging. |
40      | D6 | **Feasibility is gated on P0.** No app shell, no packaging work starts until the
compute spike returns a verdict. | This doc; `decision-rigor-norm` (NUMBERS INFORM, FOUNDATIONS
DECIDE) | P0 is the only un-gated deliverable; P1?P3 are owner-greenlit *after* P0 numbers. |
41
42      ## 3. Foundation ? research / prior art `[design]` / `[researched]`
43
44      ### 3.1 The compute reality (the crux) ? what we know vs must measure
45      - **WASB on a discrete laptop GPU is acceptable** `[verified, measured]`: on the owner's
**RTX 2070 Super Max-Q (8 GB)**; WASB inference is the pipeline driver at **~3.2?3.4× real-
time** (?3.4 GPU-min per footage-min, ~0.055 s/frame), peak VRAM ~5.7?6.0 GB; proxy (NVENC)
~0.2× RT; 4K stitch (CPU libx264) ~16 s/rally; end-to-end ?5× clip length. Source: `current-
state.md` `COST_SCALING.md` checkpoint (6 Boxhill clips, 2026-06-18).
46      - **The typical enthusiast laptop has NO discrete GPU.** A 10-min 4K clip at 3.4× RT is
~34 GPU-min *with* a 2070S. On integrated graphics / CPU-only the wall-clock is unknown and
could be 10×+ worse ? possibly **hours per clip**, which would kill the one-click promise.
47      - **Apple Silicon has no CUDA.** WASB/TrackNet weights are PyTorch; PyTorch MPS (Metal)
backend *may* run them, but op-coverage and speed on the WASB model are **unverified**
`[design]`. CPU fallback on Apple is also unverified.
48      - **Owned-model tie-in.** `OWNED_MODEL_IMPLEMENTATION_PLAN.md` already states the on-
device answer: *train in Python ? **export across the boundary** (ONNX / Core ML / ExecuTorch /
GGUF) ? a thin device shell runs the artifact with zero Python*, with the standing constraint to
**train with export-clean ops**. A smaller, exported, quantized shuttle detector is a credible
answer if the CPU/MPS numbers on the current WASB weights are bad. **This makes P0 partly a
referendum on whether the desktop product needs the owned-model track.**
49
50      ### 3.2 The de-WSL native runner is already designed-for `[verified]`
51      The detector layer was deliberately built to make this port low-regret.
[`backend/pipeline/detectors/base.py`](../backend/pipeline/detectors/base.py) documents,
verbatim, an **"Optional Linux-native path (the main de-risk goal)"**: a
`LinuxNativeRunner`/`ONNXRunner` that **"Skip[s] all wsl / _stage / conda activate, load[s]
weights locally (torch or onnx)? write[s] the exact same CSV format so
`trajectory_to_action_windows` works verbatim."** The WSL plumbing lives in a separate

```

```

`wslCommandMixin`, explicitly so a native runner "must not inherit WSL plumbing." The actual
inference math is already isolated in
[`wasb_infer.py`](../backend/pipeline/detectors/wasb_infer.py) (torch + the WASB-SBDT modules,
emitting `Frame,Visibility,X,Y`). **So the port is: lift `wasb_infer.py` out of WSL, load
weights with native torch/onnxruntime per-OS, keep the identical CSV contract.** The windowing
brain (`trajectory_to_action_windows`) and stitch are untouched.
52
53     ### 3.3 Compute backends to evaluate in P0
54     | Path | OS | Toolkit | Status |
55     |---|---|---|---|
56     | CUDA-native | Windows-native, Linux+NVIDIA | torch+CUDA (no WSL) | proven in WSL
`[verified]`; native = port |
57     | Apple MPS | macOS (Apple Silicon) | torch MPS / Core ML export | **unverified ? P0
must measure** |
58     | CPU | all (the no-GPU laptop) | torch-CPU / **onnxruntime** (often fastest on CPU) |
**unverified ? P0 must measure; the gating number** |
59     | Smaller/exported model | all | ONNX/Core ML/ExecuTorch, quantized | deferred to owned-
model track; P0 informs whether it's *needed* |
60
61     ### 3.4 App-shell prior art & licensing landscape `[researched]`
62     - **App shell options:** **Tauri** (Rust + system webview; ~3?10 MB binaries;
MIT/Apache) vs **Electron** (bundled Chromium; ~100?150 MB; MIT) vs **CLI + installer** (wrap
`generate_highlights.py`; smallest effort). All shell licenses are clean.
63     - **Backend packaging:** the Python/FastAPI backend bundles via **PyInstaller** or a
frozen venv as a local "sidecar." Packaging was already modernized ? **PEP 621 `pyproject.toml`
+ an `indexer` console entry point** shipped (`DEFERRED_HARDENING_PLAN` #7, DONE, PR #167)
`[verified]` ? so a CLI MVP has a real head start.
64     - **ffmpeg licensing (a genuine landmine):** ffmpeg is LGPL/GPL. The current stitch path
uses **libx264 (GPL)**. Redistributing that inside an installer would impose GPL on the bundle.
**Mitigation (D4):** ship an **LGPL ffmpeg build** using **openh264 (BSD)** or the OS hardware
encoder (NVENC / Apple **VideoToolbox** / Linux **VAAPI**) ? the desktop app should prefer
hardware H.264 anyway (faster, no CPU stitch tax).
65     - **Weights:** WASB-SBDT = MIT `[verified]` (`tracknet-wasb-wsl2-decision`),
redistributable.
66
67     ## 4. Design / architecture `[design]`
68
69     ### 4.1 The one-click flow (and what code each step reuses)
70     ```
71     [card inserted]
72     ? (NEW) per-OS device-insert watcher ? mount path
73     ?
74     [list videos on the card] ? glob video exts; reuse _VIDEO_EXTS from
generate_highlights.py
75     ?
76     [make on-device proxy] ? ffmpeg downscale (hardware encoder); reuse the proxy +
--detect-from split (#205)
77     ?
78     [detect: WASB native] ? (NEW) NativeWasbRunner (DetectorRunner subclass) ? NO WSL;
same CSV contract
79     ?
80     [windowing ? rallies] ? REUSE trajectory_to_action_windows + improved milestone
windowing (R1: cap=6 + R4)

```

24. user (2026-06-21T13:53:33.699Z)

```

backend\pipeline\detectors\__init__.py
backend\pipeline\detectors\fusion_features.py
backend\pipeline\detectors\rally_gate.py
backend\pipeline\detectors\audio_features.py
backend\pipeline\detectors\person_detector.py
backend\pipeline\detectors\base.py
backend\pipeline\detectors\wasb_runner.py
backend\pipeline\detectors\tracknet_runner.py
backend\pipeline\detectors\court_gate.py

```



```
backend\pipeline\detectors\stub_runner.py
backend\pipeline\detectors\wasb_infer.py
backend\pipeline\detectors\device.py
backend\pipeline\detectors\native_wasb_runner.py
```

25. user (2026-06-21T13:53:34.781Z)

Found 8 files

```
backend\pipeline\detectors\native_wasb_runner.py
backend\pipeline\detectors\device.py
backend\pipeline\detectors\wasb_infer.py
backend\pipeline\detectors\stub_runner.py
backend\pipeline\detectors\tracknet_runner.py
backend\pipeline\detectors\wasb_runner.py
backend\pipeline\detectors\base.py
backend\pipeline\detectors\__init__.py
```

26. assistant (2026-06-21T13:53:40.635Z)

Let me confirm the exact dual-env reality in the WSL runner (the hard packaging problem) and the native runner, then do final searches on Frigate's distribution/update channel and a key fact-check on Pinokio/Foocus licensing.

27. user (2026-06-21T13:53:41.506Z)

```
1      """Abstract base for detector runners (TrackNet, WASB, future native, etc.).
2
3      This completes the Detector Runner Abstraction (low-regret item 6 continuation / item 1
4      of the post-observability re-prioritization) to de-risk the WSL/conda dependency for
5      the high-accuracy shuttle trajectory substrate used by tracknet_hybrid / wasb_hybrid.
6
7      The ABC lets us:
8      - Keep the current WSL adapters working unchanged for Windows users.
9      - Swap in (or A/B test) a Linux-native implementation later with zero changes to
10     the hybrid segmenters or the model-agnostic windowing brain.
11     - Provide a single place to evolve the healthcheck / error contract and thread
12     detector readiness into per-video reports.
13
14     Contract (deliberately matches what the concrete runners + hybrids + tests already did):
15     - run_predict(video_win_path: str, output_win_dir: str) -> Optional[str]
16       Run detector, return the Windows-side path to the produced trajectory CSV
17       (e.g. ".../clip_predict.csv" or ".../clip_wasb.csv"), or None on any failure.
18       The caller (hybrid) is responsible for output dir; impls may stage the video
19       into WSL fs for perf.
20
21     - healthcheck() -> tuple[bool, str]
22       Return (ok, message). Never raises. "ok" decides whether to proceed to run_predict.
23       Message is logged (and can be surfaced to reports or UX in future).
24       Intended to be cheap (env existence, import, GPU visibility, weights present).
25
26     Existing behavior is 100% preserved. The only additions are the inheritance and
27     the explicit common type for future implementers.
28
29     How to add a new runner (for docs / future contributors):
30     1. Subclass DetectorRunner in a new or existing module under pipeline/detectors/.
31     2. Implement run_predict and healthcheck (return (bool, str) tuple).
32     3. If it produces the same CSV schema (Frame,Visibility,X,Y), reuse or re-export
33       TrackNetRunner.parse_trajectory_csv and .trajectory_to_action_windows (they are
34       deliberately static and model-agnostic). If the new detector needs different
35       windowing, it can implement its own or we generalize later.
36     4. Lazy-import heavy native deps inside the methods (so importing the segmenter
37       registry never pulls torch/tf on a machine that doesn't have them).
38     5. Update the two hybrid segmenters (or introduce a detector= choice) to accept
39     DetectorRunner instances (they already do the right thing via duck-typing + the
```

```

40     type hints we add here).
41 6. Add unit tests that exercise the contract (isinstance, healthcheck shape, run
42    returns str|None) using mocks for the external bits ? see test_tracknet_runner.py.
43 7. No change to eval/calibration code paths (they consume the CSV + the static
44    windowing helpers directly).
45
46 Optional Linux-native path (the main de-risk goal):
47 A LinuxNativeRunner (or ONNXRunner) would:
48 - Skip all wsl / _stage / conda activate.
49 - Load weights locally (torch or onnx) or call a native binary.
50 - Write the exact same CSV format so trajectory_to_action_windows works verbatim.
51 - healthcheck can just check "weights file exists and torch importable + CUDA?".
52 This removes the WSL requirement for Linux users / future SaaS workers while
53 keeping the exact same candidate quality characteristics (or better, once native
54 weights are available).
55
56 Error surfacing into reports:
57 The healthcheck message is already logged by the hybrids on failure. The reporting
58 harvest sees the outcome as "0 candidates" + detector name. When we wire richer
59 segmenter Results (or pass detector_health into emit_indexer_report context), the
60 (ok, msg) can be recorded explicitly under segmentation.details or a new detector

```

28. user (2026-06-21T13:53:43.966Z)

```

36:     conda_env: str = "wasb"
56:         conda_env=w.get("conda_env", "wasb"),
74:         cmd = (f"conda activate {self.cfg.conda_env} && "
162:             f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "

```

29. user (2026-06-21T13:53:50.153Z)

Web search results for query: "Frigate release channels stable beta Docker tag versioning auto-update Watchtower how users update"

Links: [{"title": "Automatically updating frigate when there's a new release, using watchtower · blakeblackshear/frigate · Discussion #6769", "url": "https://github.com/blakeblackshear/frigate/discussions/6769"}, {"title": "Updating | Frigate", "url": "https://docs.frigate.video/frigate/updating/"}, {"title": "Releases · blakeblackshear/frigate", "url": "https://github.com/blakeblackshear/frigate/releases"}, {"title": "Is 16 still the 'stable' release? · blakeblackshear/frigate · Discussion #22827", "url": "https://github.com/blakeblackshear/frigate/discussions/22827"}, {"title": "Add Docker Tags for beta versions · Issue #2706 · blakeblackshear/frigate", "url": "https://github.com/blakeblackshear/frigate/issues/2706"}, {"title": "The official docker image tags for the current stable version are · blakeblackshear/frigate · Discussion #10093", "url": "https://github.com/blakeblackshear/frigate/discussions/10093"}]

Based on the search results, here's what I found about Frigate's release channels, Docker tag versioning, and how users update:

Release Channels and Docker Tags

Frigate offers a stable tag (e.g., ghcr.io/blakeblackshear/frigate:stable) that always points to the latest stable release after pulling. Additionally, the current Frigate docs tags include stable, stable-standard-arm64, and stable-tensorrt.

Manual Updating

Users can manually update by editing their docker-compose.yml file. Users can edit their docker-compose.yml file to specify the desired version tag (e.g., 0.17.2 instead of 0.16.4). Since users using the stable tag don't need to update the tag in their docker-compose.yml manually, they just need to pull the image and the stable tag will give them the latest version.

Auto-Updating with Watchtower

Users can set up watchtower to automatically update Frigate when the stable release changes. The watchtower service requires proper docker-compose configuration with labels and scope settings to enable automatic updates.

The documentation and community discussions show that Frigate supports multiple update methods, from manual version tag specification to automated updates via Watchtower for users who prefer hands-off updates.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

30. assistant (2026-06-21T13:53:55.780Z)

Confirmed the hard packaging problem: `wasb\_runner.py:74,162` literally runs `conda activate {conda\_env}` (default `wasb`) inside WSL ? a separate torch-1.11/cu113 env. The native runner exists to escape it. Let me do two final targeted searches to round out the licensing facts (Pinokio license, Fooocus license, Tauri vs Electron, and the "uv" pattern for torch index-url) which are load-bearing for our conclusion.

31. user (2026-06-21T13:54:04.314Z)

Web search results for query: "Pinokio license open source AGPL MIT; Fooocus license GPL; can redistribute commercially"

Links: [{"title": "MIT license - pinokiocomputer/pinokio", "url": "https://github.com/pinokiocomputer/pinokio/blob/main/LICENSE"}, {"title": "An Open-Source AI Tool for Voice Generation ? Both.org", "url": "https://www.both.org/?p=11437"}, {"title": "Simplifying local AI with Pinokio ? The Future is Open", "url": "https://donwatkins.info/2025/07/30/simplifying-local-ai-with-pinokio/"}, {"title": "Pinokio", "url": "https://beta.pinokio.co/"}, {"title": "Tired of complex AI installs? Try Pinokio | We Love Open Source ? All Things Open", "url": "https://allthingsopen.org/articles/pinokio-facefusion-local-ai-playground"}, {"title": "Open Source Licenses: Which One Should You Pick? MIT, GPL, Apache, AGPL and More (2026 Guide) - DEV Community", "url": "https://dev.to/juanisidoro/open-source-licenses-which-one-should-you-pick-mit-gpl-apache-agpl-and-more-2026-guide-p90"}, {"title": "Pinokio Alternatives - Explore Similar Software | AlternativeTo", "url": "https://alternativeto.net/software/pinokio/"}, {"title": "Pinokio makes installing ComfyUI a breeze", "url": "https://www.xda-developers.com/pinokio-how-to/"}, {"title": "GitHub - pinokiocomputer/pinokio: AI Browser . GitHub", "url": "https://github.com/pinokiocomputer/pinokio"}, {"title": "Open Source Licenses Explained: AGPL, MIT, GPL, Apache 2.0, and What Each Means for Your Project in 2026", "url": "https://www.opensourcealternatives.to/blog/open-source-license-guide"}]

Links: [{"title": "Fooocus/LICENSE at main · llllyasviel/Fooocus", "url": "https://github.com/lllyasviel/Fooocus/blob/main/LICENSE"}, {"title": "Fooocus-AI-Image-Generator/LICENSE at main · kdjayakody/Fooocus-AI-Image-Generator", "url": "https://github.com/kdjayakody/Fooocus-AI-Image-Generator/blob/main/LICENSE"}, {"title": "Commercial use of images generated using Fooocus · llllyasviel/Fooocus · Discussion #1957", "url": "https://github.com/lllyasviel/Fooocus/discussions/1957"}, {"title": "Fooocus download | SourceForge.net", "url": "https://sourceforge.net/projects/fooocus.mirror/"}, {"title": "Fooocus Pricing & Review 2026 | Coda One", "url": "https://www.codaone.ai/tools/fooocus/"}, {"title": "Fooocus - Advanced image focus selection and depth estimation tool | Professional image editing with AI technology implementation", "url": "https://www.fooocus.cc/"}, {"title": "fooocus/LICENSE.md at main · ai-dock/fooocus", "url": "https://github.com/ai-dock/fooocus/blob/main/LICENSE.md"}, {"title": "Fooocus - Focus on Prompting and Generating | AI Image Generation Tool", "url": "https://fooocus.xyz/"}, {"title": "Fooocus ? Grokipedia", "url": "https://grokipedia.com/page/Fooocus"}, {"title": "LICENSE · LykosAI/GPT-Prompt-Expansion-Fooocus-v2 at main", "url": "https://huggingface.co/LykosAI/GPT-Prompt-Expansion-Fooocus-v2/blob/main/LICENSE"}]

Links: [{"title": "Open-source license - Wikipedia", "url": "https://en.wikipedia.org/wiki/Open-source_license"}, {"title": "Comparison of free and open-source software licenses - Wikipedia", "url": "https://en.wikipedia.org/wiki/Comparison_of_free_and_open-

```
source_software_licenses"},"title":"Licenses ? Open Source
Initiative","url":"https://opensource.org/licenses"},"title":"Understanding Open Source
Licensing - Semaphore","url":"https://semaphore.io/blog/open-source-licensing"},"title":"Open
Source Software Licensing ? Georgia Tech Open Source Program
Office","url":"https://ospo.cc.gatech.edu/open-source-software-licensing/"},"title":"The Legal
Side of Open Source | Open Source
Guides","url":"https://opensource.guide/legal/"},"title":"Frequently Answered Questions ? Open
Source Initiative","url":"https://opensource.org/faq"},"title":"Understanding Open Source and
Free Software Licensing By Andrew M. St. Laurent","url":"https://people.debian.org/~dktrkranz/le
gal/Understanding%20Open%20Source%20and%20Free%20Software%20Licensing.pdf"},"title":"Open
Source Software and Licensing ? A Primer | Kynetics - Kynetics Technical
Notes","url":"https://technotes.kynetics.com/2018/open-source-software-and-licensing/"]]
```

Based on my web searches, here's what I found about the licenses and commercial redistribution:

Pinokio License

Pinokio is open source with an MIT license. The search results do not mention AGPL for Pinokio itself.

Foocus License

Foocus is open-source under the GPLv3 license (not AGPL as mentioned in your query).

Commercial Redistribution

Regarding whether these can be redistributed commercially:

****Pinokio (MIT License):**** MIT grants anyone the right to use, copy, modify, merge, publish, distribute, sublicense, and sell software built on MIT-licensed code. The only requirement is that recipients receive the original copyright notice and license text. This means commercial redistribution is allowed.

****Foocus (GPL License):**** The GPL allows commercial use, but if you distribute the product, you must provide the corresponding GPL source code to your recipients. Commercial use is permitted, but commercial distribution has restrictions.

More generally, all Open Source software can be used for commercial purpose; the Open Source Definition guarantees this. You can even sell Open Source software. However, if you receive software under an Open Source license, you can always use that software for commercial purposes, but that doesn't always mean you can place further restrictions on people who receive the software from you. In particular, copyleft-style Open Source licenses require that, in at least some cases, when you distribute the software, you must do so under the same license you received it under.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

32. user (2026-06-21T13:54:05.649Z)

Web search results for query: "uv python package manager torch install --index-url CUDA pyproject extra-index pinning standalone python download"

Links: [{"title":"Using uv with PyTorch | uv","url":"https://docs.astral.sh/uv/guides/integration/pytorch/"},"title":"How to Install PyTorch with uv | pydevtools","url":"https://pydevtools.com/handbook/how-to/how-to-install-pytorch-with-uv/"},"title":"Issues creating a cuda-enabled pytorch environment with UV · Issue #7202 · astral-sh/uv","url":"https://github.com/astral-sh/uv/issues/7202"},"title":"Managing PyTorch CPU/CUDA Versions per Environment with uv Part 2 | MJUN Tech Note","url":"https://mjunya.com/en/posts/2024-10-22-python-uv-pytorch-2/"},"title":"My Struggle Setting Up a PyTorch (CUDA) Environment with uv via Vibe Coding: The Trap of Non-Existent Versions and Python 3.12??AI????????","url":"https://note.com/daisukeminagawa/n/nc16e5895996b?"}]

```
hl=en"}, {"title": "Simplifying PyTorch Environment Setup with uv: A Unified Approach for Cross-Platform and CPU/GPU Support", "url": "https://zenn.dev/haru256/articles/6ded722b409d13?locale=en"}, {"title": "GitHub - BaoNguyen6742/uv-install-torch: Tutorial to install torch/pytorch with cuda using uv · GitHub", "url": "https://github.com/BaoNguyen6742/uv-install-torch"}, {"title": "Can't install e.g. new `packaging` and CUDA version of torch simultaneously · Issue #2683 · astral-sh/uv", "url": "https://github.com/astral-sh/uv/issues/2683"}]
```

I've performed the web search for your query. Here are the key findings about using uv (a Python package manager) with PyTorch and CUDA:

Key Concepts

uv supports PyTorch installation with different accelerators (CPU-only vs CUDA) through optional dependency groups specified in `pyproject.toml`. Different accelerator builds are published to different indexes; for example, CPU builds are on <https://download.pytorch.org/whl/cpu>, while CUDA 13.0 builds are on <https://download.pytorch.org/whl/cu130>.

Configuration Approach

To install a specific CUDA build across platforms, you need to define a PyTorch index in `pyproject.toml` and route packages to it with `tool.uv.sources`. The `explicit` flag indicates that the index should only be used for packages that explicitly specify it in `tool.uv.sources`; if `explicit` is not set, other packages may be resolved from the index. For PyTorch indexes, this setting should be enabled to restrict the index-specified URL from being used when searching for other packages.

Available Options

Available PyTorch backends follow the URL pattern <https://download.pytorch.org/whl/{backend}>, where `{backend}` is one of: `cpu`, `cu118`, `cu126`, `cu128`, `cu130`, `rocm6.4`, `xpu`.

Command-Line Installation

When using `uv add` with torch, you can use `--extra-index-url` to resolve dependencies and install the CUDA version, as `--index-url` is not saved in `pyproject.toml`.

This configuration allows for flexible PyTorch installation across different platforms and hardware setups using uv's index and extras system.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

33. assistant (2026-06-21T13:54:16.971Z)

I now have everything I need. Key load-bearing facts confirmed:

- **Our hard problem is real and in-code**: `wasb_runner.py:74,162` runs `conda activate {conda_env}` (default `wasb`) inside WSL ? a separate torch-1.11/cu113 env, while the main engine is torch-2.6+cu124. native_wasb_runner.py` exists as the escape hatch (in-process torch, no WSL/conda).`
- **Pinokio = MIT** (license-clean, and it solves *exactly* our problem: per-app isolated conda/venv + GPU libs + bundled ffmpeg with one click).
- **ComfyUI desktop = GPL-3.0** (architecture is the best precedent: Electron shell + bundled **uv** that installs torch on first run + NSIS installer + auto-update; but the shell itself is GPL ? copy the *pattern*, not the code).
- **Foocus = GPL-3.0**, embedded-python + run.bat + first-run weight download.
- **uv** natively expresses the `--index-url` torch problem our pyproject.toml says PEP 621 `can't ? `[tool.uv.sources]` + `[tool.uv.index]` with `explicit=true`. This is the single most directly applicable technical pattern for us.`
- Frigate ships ffmpeg *inside* the Docker image at `/usr/lib/ffmpeg` ; distributes via Docker tags (stable` ) + Home Assistant add-on; updates via image re-pull / Watchtower; Windows is`

unofficial (WSL2 + Docker Desktop, GPU passthrough hard) ? a cautionary tale for our non-tech Windows user.

Let me record a chapter marker and return the structured research.

34. user (2026-06-21T13:54:18.647Z)

Chapter marked: "Batteries-included packaging research". Continue your current work.

35. user (2026-06-21T13:56:18.316Z)

Structured output provided successfully

36. assistant (2026-06-21T13:56:24.571Z)

Research complete. I returned the RESEARCH schema covering five distribution patterns (Pinokio launcher, ComfyUI-desktop Electron+uv, embedded-Python portable ZIP, Frigate Docker, and LM Studio/Jan/GPT4All bundled-engine), each with how it handles the GPU/torch + ffmpeg + weights problems, install UX, and updates ? all grounded in web sources and our actual repo files (notably the in-code dual-env at `wasb\_runner.py:74,162` and the `uv` torch `--index-url` fix for the blocker documented in `pyproject.toml:8-11`).

The recommendation is a layered, profile-matched strategy rather than one installer, with the top-leverage prerequisite being to finish `native\_wasb\_runner.py` so the WASB conda sidecar collapses into a single torch env.